WNDRVR
SUPPORT NETWORK

VXWORKS 7 NETWORK DRIVER DEVELOPER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Network Driver Developer's Guide, SR0650

2 July 2020

1. NETWORK DEVICE DRIVER OVERVIEW

1. About VxWorks Network Drivers

VxBus provides support for the development of VxWorks® network device drivers. Before beginning your development, you must understand the network device driver ecosystem in VxWorks.

In VxWorks 7, all network drivers are VxBus-enabled. For more information on VxBus, see the VxWorks 7 BSP and Driver Guide.

The majority of VxWorks networks use Ethernet as the transmission media. Wind River also supports VxWorks network drivers and configurations for shared memory, serial network transports, and wireless Ethernet drivers. While other network transports are possible, VxBus network driver development is primarily focused on Ethernet as the transport.

Most modern VxWorks Ethernet drivers are split into two parts: a MAC driver and a PHY driver. Together, the MAC driver and the PHY driver manage the data link layer within the OSI model. The MAC sub-layer of the data link layer manages protocol access to the physical network medium. This sub-layer deals with extracting data from the wire to send to the protocol, gaining access to the wire to send protocol data, and certain other aspects regarding the transmission of already packetized protocol data. The PHY sub-layer deals with frame synchronization, flow control, error checking, and other aspects of manipulating individual bits and bytes during transmission.

Seven Layer OSI Model Overview on page 1 VxWorks uses a model of network software known as the *seven layer OSI model*, published by the Open Systems Interconnection (OSI) organization.
MUX Overview on page 2
Terminology on page 2 Before developing a VxWorks network driver, understand the relevant VxBus network driver terminology.

1.1. Seven Layer OSI Model Overview

VxWorks uses a model of network software known as the *seven layer OSI model*, published by the Open Systems Interconnection (OSI) organization.

Open Systems Interconnection (OSI) is an organization that defines and publishes a model of network software. The published model consists of seven layers. This definition of layers is used throughout Wind River documentation when discussing network stacks and drivers.

Figure 1. Seven Layer OSI Model

Layer 7 - Application
Layer 6 - Presentation
Layer 5 - Session
Layer 4 – Transport (for example TCP and UDP)
Layer 3 - Network
Layer 2 – Data Link (MAC and LLC)
Layer 1 - Physical

Parent topic: About VxWorks Network Drivers on page 1

Related information
MUX Overview on page 2
Terminology on page 2

1.2. MUX Overview

Within VxWorks, network drivers are generally decoupled from the networking protocols used in the VxWorks system. This is done by a layer of software between the protocol and the driver. In VxWorks, this is called the multiplexor (MUX).

The MUX sits between the network (OSI layer 3) and the data link layer (OSI layer 2). The purpose of the MUX is to decouple the network driver from the network protocols, thus making the network driver and network protocols nearly independent from each other. This independence makes it easier to add new drivers or protocols. If you add a new VxWorks network driver, all existing MUX-based protocols can use the new driver. Likewise, if you add a new MUX-based protocol, any existing network driver can use the MUX to access the new protocol.

For example, after receiving a packet, the MAC driver does not directly access any structure within the network protocol. Instead, the driver calls a MUX-supplied function that handles the details of passing the data up to the network protocol. For more information on the MUX, see MUX Interface on page 20.

Parent topic: About VxWorks Network Drivers on page 1

Related information
Seven Layer OSI Model Overview on page 1
Terminology on page 2

1.3. Terminology

Before developing a VxWorks network driver, understand the relevant VxBus network driver terminology.

The following terms are used to discuss VxWorks networking and VxBus network drivers in the context of device driver development:

enhanced network driver (END)

In earlier releases of VxWorks, network drivers are commonly referred to as *END drivers*. In the current release, this term is deprecated in favor of media access controller (MAC) driver. However, the term *END driver* is used to refer to the network driver's interface with the MUX. The MUX/END interface is usually provided by a MAC driver (when the MAC and PHY drivers are separate).

media access controller (MAC)

MAC devices are commonly thought of as network interfaces. In the context of VxWorks device driver development, the term *media access controller* and the acronym *MAC* are used to describe network interfaces. In addition, the term *MAC driver* is used to describe network interface drivers.

multiplexor (MUX)

In VxWorks, network drivers are written such that they are largely decoupled from the networking protocol that is being used. This is done using a layer of software between the protocol and the drive called the multiplexor (MUX).

Parent topic: About VxWorks Network Drivers on page 1

Related information

Seven Layer OSI Model Overview on page 1
MUX Overview on page 2

2. NETWORK INTERFACE DRIVERS

1. MAC Driver Overview

Network interface drivers are generally referred to as media access controller (MAC) drivers in VxWorks. These drivers must provide specific functionality in a VxWorks system.

To understand some terminology, the MAC is the physical device for which you are writing a VxBus device driver. Because your hardware can have multiple MAC chips on it, each one uses a single VxBus driver instance (a device and driver pairing).

The functional units of a MAC driver include the following:

- a receiver
- a transmitter
- a command and control module

MAC drivers must also handle interrupts in a way that is compatible with VxWorks.

Reception

The driver receiver is composed of functions that execute an algorithm to do the following:

- Accept incoming frames from a DMA engine.
- Pass the incoming frames to the MUX.
- Provide the DMA engine with a continuous supply of DMA buffers.

A MAC device notifies the driver of received frames by means of a device-generated interrupt. The driver does not directly service incoming frames in the interrupt context but defers the work to a function running in a task context. The task that the driver uses to deliver received packets to the MUX must be one of the network tasks (**tNet0**, **tNet1**, and so forth). Drivers that support multiple receive queues can deliver packets received on different queues in different network tasks.

For VxBus MAC drivers, each instance of the MAC driver has a private buffer pool into which incoming DMAs are directed. A MAC driver loans individual buffers from its pool to the stack. After having been loaned to the network stack, there is no guarantee that any individual buffer will be returned to the driver in a particular order or in a timely manner.

Transmission

The driver transmitter is composed of functions that execute an algorithm to do the following:

- Accept packets from the MUX and transfer them to the device's transmit DMA engine.
- Reclaim the resources associated with a transmitted packet.

A protocol requests that a MAC driver transmit a frame by calling the mux2send( ), muxTkSend( ), or muxSend( ) function, which in turn calls the driver's registered send function. Sends can occur at any time, and may occur before previous sends are complete.

Resource reclamation of DMA buffers and control structures is generally stimulated by a device-generated, transmit-packet-complete interrupt. This interrupt announces that the device has sent a complete frame and that the driver can now return the corresponding memory resources back to their pools of origin. Actual freeing of the memory resources is generally deferred to a task context rather than being done in the transmit-complete ISR. Drivers can also free resources that are associated with earlier completed packet transmissions while in their send function setting up subsequent transmissions.

Command and Control Module

The command and control module provides configuration, initialization, and control interfaces for the device. The MAC driver command and control module is the part of the driver that obtains the driver configuration parameters from the device tree blob (DTB) (for ARM, PowerPC, and RISC-V systems) or from the Intel Architecture device table (IADT) (for Intel Architecture systems), quiesces the device, and configures the device in the prescribed mode. It incorporates the driver's load, unload, start, stop, and ioctl functions, as well as functions for querying and modifying the multicast filter. In essence, the driver's command and control module provides the driver's external interface, with the exception of send and receive. The driver ISR is considered a part of the driver command and control module. Drivers can have more than one ISR.

Network Driver Interrupts

There are several limitations on network interrupts in VxWorks. These limitations impact the way drivers are written.

The driver's interrupt handling code generally serves three functions. These functions include:

- Handling receive interrupts.
- Returning resources to the pool after a packet is transmitted.
- Handling link status changes or various error conditions.

There are two common configurations of interrupts for network devices. Network devices can provide a single interrupt line for all types of interrupts or they can provide one interrupt line each for transmit events, receive events, and error events.

When your network device provides only a single interrupt line for all types of interrupts, only a single ISR can be called to service all types of interrupts. When this ISR is called, it must check a register to see what type of action is required. The ISR reads the device register and invokes the appropriate functions to handle each type of event that has occurred .

The task-level functions for each type of interrupt must process all of the work that is available for that particular type. (See Network Interface Driver Receive Handling on page 34

> Note: In some cases, a single task-level function handles all of the possible events.

2. VxBus Device Driver Methods for MAC Drivers

Network interface device drivers (MAC drivers) must implement several VxBus methods.

Your MAC driver must, at a minimum, implement the following VxBus device driver methods: {devAttach}( ), {vxbDevDetach}( ), {devProbe}( ), and {miiMediaUpdate}( ).

> Note: The use of curly brackets with method names indicates that the string within the pair of curly brackets is user-defined.

> Note: Some MAC drivers also implement the {miiRead}( ) and {miiWrite}( ) methods. However, for new driver development, these methods should be implemented in your MDIO driver. For more information, see MDIO Drivers on page 55.

The media independent interface (MII) driver methods provide a means of communication between a MAC driver and a PHY driver. The full interface involves driver methods provided by MAC drivers and invoked by PHY drivers and additional driver methods provided by PHY drivers and invoked by MAC drivers. For information on the driver methods provided by PHY drivers, see Device Driver Methods for PHY Drivers on page 48

{devAttach}( )

The {devAttach}( ) driver method is called by VxBus (from the parent device, such as the flattened device tree (FDT) bus or the PCI bridge) to create an instance of the device. This involves initialization of internal data structures of the driver, initialization of the device hardware, and resource allocation (memory/IO access and interrupts). In addition, this method also determines the PHY ID of the connected PHY and then creates a PHY child device. When the device uses DMA, you must also initialize and enable DMA. Finally, this method connects the device to the MUX.

```
LOCAL STATUS devAttach
    (VXB_DEV_ID pDev)
    {
        dev_DRV_CTRL * pDrvCtrl;
        VXB_RESOURCE_ADR * pResAdr = NULL;
        VXB_RESOURCE *     pRes;
        VXB_DEV_ID pVxbDev;
        PHY_HARDWARE * pPhyDev;

    /* get device info  - if PCI device, read PCI bus, otherwise read device config info
        From IADT/FDT  */

    /* Get the device type. */
        VXB_PCI_BUS_CFG_READ (pDev, PCI_CFG_DEVICE_ID, 2, pDrvCtrl->geiDevId);

    /* Get the revision code */
        VXB_PCI_BUS_CFG_READ (pDev, PCI_CFG_REVISION, 1, pDrvCtrl->geiRevId);

    /* or read from FDT/IADT  (if not PCI device) */
        prop = vxFdtPropGet(pFdtDev->offset, "my_prop_name", &proplen);

    /* Gain memory access to device register (if applicable) */
        pRes = vxbResourceAlloc(pDev, VXB_RES_MEMORY, 0);
        pResAdr = (VXB_RESOURCE_ADR *)pRes->pRes;
        pDrvCtrl->devHandle = pResAdr->pHandle;
        pDrvCtrl->devBar = (void *)pResAdr->virtual;

    /* Gain IO access to device register  (if applicable) */
        pRes = vxbResourceAlloc(pDev, VXB_RES_IO, 0);
        pResAdr = (VXB_RESOURCE_ADR *)pRes->pRes;
        pDrvCtrl->devIoHandle = pResAdr->pHandle;
        pDrvCtrl->devIoBar = (void *)pResAdr->virtual;

    /* Reset the chip  - initialize the hardware */
        devReset (pDev);

    /* Initialize DMA for the device (Tx and RX buffers)
        vxbDmaBufTagCreate( …)
        vxbDmaBufMemAlloc( …)
        vxbDmaBufMapCreate(…)

    /* create the PHY device - set the pPhyDev structure */
        pVxbDev = NULL;
        vxbDevCreate (&pVxbDev);

    /* Save the PHY address */
        pPhyDev->phyAddr = (UINT8)pDrvCtrl->geiMiiPhyAddr;
```

```
    /* find the PHY ID, this will be used by the PHY driver’s probe method */
        MII_READ (pDev, pPhyDev->phyAddr, MII_PHY_ID1_REG, &pPhyDev->phyId1);

    /* store PHY ID so PHY driver could retrieve it later */
        vxbDevClassSet(pVxbDev, VXB_BUSID_MII);
        vxbDevIvarsSet(pVxbDev, pPhyDev);
        vxbDevAdd(pDev, pVxbDev)  /* this will call “probe” and “attach” methods in the PHY drive
r */
        pDrvCtrl->devMiiBus = pVxbDev;

    /* Connect the device to the MUX */
        devMuxConnect (pDev, NULL);

    }
```

You must implement a devMuxConnect( ) function (the name is up to you) in your MAC device driver. It must be called at the end of the devAttach( ) function.

Most VxBus initialization occurs before the MUX is fully initialized. Therefore, the usual muxDevLoad( )/muxDevStart( ) sequence must be deferred until the networking subsystem is ready.

```
LOCAL void devMuxConnect(VXB_DEV_ID   pDev, void *unused)
{
    GEI_DRV_CTRL *pDrvCtrl;

    pDrvCtrl = vxbDevSoftcGet(pDev);

    pDrvCtrl->myDevMuxDevCookie = muxDevLoad (pDrvCtrl->unit,
                                  myDevEndLoad, "", TRUE, pDev);

    if (pDrvCtrl->myDevMuxDevCookie != NULL)
        (void)muxDevStart (pDrvCtrl->myDevMuxDevCookie);

}
```

You must also implement a myDevEndLoad( ) END load function (the name is up to you), which is passed as a callback to the muxDevLoad( ) function. This function initializes the END interface (that is, the connection between MAC driver and the VxBus network driver infrastructure) associated with this device. The load string first parameter is an empty string. The second argument is a pointer to the VxBus device instance associated with this device. This function returns the device name (if the initial string is empty) because this is the behavior expected by the MUX. The MUX invokes this function twice; once to obtain the device name, and then again to create the actual **END_OBJ** instance (device-driver pairing). When this function is called the second time, it initializes the END object, performs MIB2 setup, allocates a buffer pool, and initializes the supported END capabilities.

```
LOCAL END_OBJ * geiEndLoad (char * loadStr, void * pArg)
{
    GEI_DRV_CTRL *pDrvCtrl;
    VXB_DEV_ID pDev;

    if (loadStr[0] == 0)
    {
        bcopy (ENET_NAME, loadStr, sizeof(ENET_NAME));
        return (NULL);
     }

    pDev = pArg;
    pDrvCtrl = (ENET_DRV_CTRL *)vxbDevSoftcGet (pDev);
```

```
    END_OBJ_INIT (&pDrvCtrl->myDevEndObj, NULL, MY_DEV_NAME,
        (int)pDrvCtrl->unit, &myDevNetFuncs, "My Dev VxBus END Driver");

   (void) endM2Init (&pDrvCtrl->myDevEndObj, (long)M2_ifType_ethernet_csmacd,
        pDrvCtrl->myDevAddr, ETHER_ADDR_LEN, ETHERMTU, 100000000,
        IFF_NOTRAILERS | IFF_SIMPLEX | IFF_MULTICAST | IFF_BROADCAST);

  /* allocate a buffer pool */
    pDrvCtrl->myDevMaxMtu = ENET_MTU;
    if (endPoolCreate (ENET_TUPLE_CNT, &pDrvCtrl->myDevEndObj.pNetPool);

  /* Set up polling stats. */
    pDrvCtrl->myDevEndStatsConf.ifPollInterval = sysClkRateGet();
    ...
    pDrvCtrl->myDevEndStatsConf.ifValidCounters = (END_IFINUCASTPKTS_VALID |
            END_IFINMULTICASTPKTS_VALID | END_IFINBROADCASTPKTS_VALID |
            END_IFINOCTETS_VALID | END_IFINERRORS_VALID | END_IFINDISCARDS_VALID |
            END_IFOUTUCASTPKTS_VALID | END_IFOUTMULTICASTPKTS_VALID |
            END_IFOUTBROADCASTPKTS_VALID | END_IFOUTOCTETS_VALID |
            END_IFOUTERRORS_VALID);

/* set up capabilities */

    pDrvCtrl->myDevCaps.cap_available = IFCAP_VLAN_MTU;
    pDrvCtrl->myDevCaps.cap_enabled = IFCAP_VLAN_MTU;

    return (&pDrvCtrl->myDevEndObj);
}
```

This function also passes the **myDevNetFuncs[]** function table to the MUX. In the MAC device driver, you must implement the following NET_FUNCS functions, which are required by MUX facility:

```
{
    STATUS (*start) (END_OBJ*);            /* Driver's start func. */
    STATUS (*stop) (END_OBJ*);             /* Driver's stop func. */
    STATUS (*unload) (END_OBJ*);                   /* Driver's unload func. */
    int (*ioctl) (END_OBJ*, int, caddr_t);    /* Driver's ioctl func. */
    STATUS (*send) (END_OBJ* , M_BLK_ID);          /* Driver's send func. */
    STATUS (*mCastAddrAdd) (END_OBJ*, char*); /* Driver's mcast add func. */
    STATUS (*mCastAddrDel) (END_OBJ*, char*); /* Driver's mcast delete func. */
    STATUS (*mCastAddrGet) (END_OBJ*, MULTI_TABLE*); /* Driver's mcast get fun. */
    STATUS (*pollSend) (END_OBJ*, M_BLK_ID);  /* Driver's polling send func. */
    STATUS (*pollRcv) (END_OBJ*, M_BLK_ID);   /* Driver's polling recv func. */
    M_BLK_ID (*formAddress) ();         /* Driver's addr formation func. */
    STATUS (*packetDataGet) ();         /* Driver's packet data get func. */
    STATUS (*addrGet) (M_BLK_ID, M_BLK_ID, M_BLK_ID, M_BLK_ID, M_BLK_ID); /* Driver's packet addr
get func. */
} NET_FUNCS;
```

{vxbDevDetach}( )

The {vxbDevDetach}( ) driver method requests that an instance be shut down. This can occur if your VxBus instance is terminated, or if the driver is unloaded. When an unlink event occurs, you must do the following:

WNDRVR

- Shut down and unload the driver interface associated with this device instance.
- Release all the resources allocated during instance creation, such as vxbDma memory and maps.
- Shut down and unload all interrupt handles associated with this instance.

If the driver created an MII bus, you must also destroy the child MII bus and PHY devices.

{devProbe}( )

The {devProbe}( ) driver method is used by VxBus to check if the driver must be used to control a particular device instance of the same class as the driver. Exactly what the {devProbe}( ) method does depends upon the driver class. For FDT drivers, it attempts to match the device driver's name as appears in the flattened device tree (FDT). For PCI-based MAC drivers, the {devProbe}( ) method typically calls vxbPciDevMatch( ) to compare the instance being probed against a table of the driver's supported PCI **deviceID** / **vendorID** combinations.

{miiMediaUpdate}( )

The {miiMediaUpdate}( ) method has the same signature as the following stub routine:

```
LOCAL STATUS miiMediaUpdate
    (
    VXB_DEV_ID pDev
    )
    {
    return (OK);
    }
```

Note: In Wind River provided drivers, the function that implements the {miiMediaUpdate}( ) method typically uses a naming convention like *dev* LinkUpdate( ), where *dev* is an abbreviation of the device type. For example, geiLinkUpdate( ) for the gei driver.

This method is invoked by the MII bus layer whenever the **miiBusMonitor** task detects a link state change, which is either a transition from link up to link down or from link down to link up. Normally, the **miiBusMonitor** task checks for link events every two seconds.

The **pDev** argument is a pointer to the MAC driver's instance handle. The MAC driver uses this method to perform any operations that are required when a link state change occurs. This can include setting the MAC speed to match the link speed, enabling or disabling full duplex mode, or configuring flow control. (If the hardware is designed to handle link state changes automatically and does not need any software assistance, your driver can omit these steps.) The driver can query the current link state using the miiBusModeGet( ) function.

The {miiMediaUpdate}( ) method is also typically used by drivers to notify bound protocols of link state changes. You must determine the new link state. If the device instance is administratively up (that is, not stopped), you must announce the change to any bound protocols. You can notify the protocols by posting a job to the driver's job queue for the device. The job is to execute either the muxLinkUpNotify( ) or muxLinkDownNotify( ) function. This is shown in the following example from the etsec driver:

```
/* If status went from down to up, announce link up. */
    if (pDrvCtrl->etsecCurStatus & IFM_ACTIVE && !(oldStatus & IFM_ACTIVE))
        jobQueueStdPost (pDrvCtrl->etsecJobQueue, NET_TASK_QJOB_PRI,
                         muxLinkUpNotify, &pDrvCtrl->etsecEndObj,
                         NULL, NULL, NULL, NULL);

    /* If status went from up to down, announce link down. */
    else if (!(pDrvCtrl->etsecCurStatus & IFM_ACTIVE) && oldStatus &
                IFM_ACTIVE)
            jobQueueStdPost (pDrvCtrl->etsecJobQueue, NET_TASK_QJOB_PRI,
```


WNDRVR

```
                                muxLinkDownNotify, &pDrvCtrl->etsecEndObj,
                                NULL, NULL, NULL, NULL);
```

In addition, this method updates MIB information (RFC 1213 **ifSpeed** member and RFC 2233 **ifHighSpeed** member) so that the correct interface speed values are reported to SNMP queries. Assuming the variable speed has the current interface speed in bits per second, and **pEnd** points to **END_OBJ**, the following code can accomplish this:

```
pEnd->mib2Tbl.ifSpeed = speed;
if (pEnd->pMib2Tbl != NULL)
    {
    pEnd->pMib2Tbl->m2Data.mibIfTbl.ifSpeed = speed;
    pEnd->pMib2Tbl->m2Data.mibXIfTbl.ifHighSpeed =
        (speed + 500000) / 1000000;
    }
```

3. MAC Driver Header File Usage

MAC drivers typically include several header files. Header files related to the network stack interface, VxBus utilities, and MII bus utilities are included.

Note: MAC drivers are not required to include all of the header files listed in this section.

MAC drivers typically include the following header files related to the network stack interface:

```
#include <muxLib.h>
#include <netLib.h>
#include <netBufLib.h>
#include <etherMultiLib.h>
#include <end.h>
#define END_MACROS
#include <endLib.h>
#include <endMedia.h>
```

Header files for VxBus utilities are generally included as follows:

```
#include <hwif/vxBus.h>
#include <subsys/int/vxbIntLib.h>
#include <vxbMiiLib.h>

/*if the DMA library is needed */
#include <hwif/util/vxbDmaBufLib.h>

/*if the FDT library is needed */
#include <hwif/buslib/vxbFdtLib.h>

/* for PCI drivers */
#include <hwif/buslib/pciDefines.h>
#include <hwif/buslib/vxbPciLib.h>
#include <hwif/buslib/vxbPciMsi.h>
```

Header files for MII bus utilities are typically included as follows:

WNDRVR

```
#include <vxbMiiLib.h>
```

If you are working with ARM or PowerPC target hardware, the following header file is required for certain functions related to the flattened device tree (FDT):

```
#include <miiBus.h>
```

4. MAC Driver DTS Configuration

For ARM and PowerPC systems, you must configure the device tree source (DTS) files with the required configuration information for your MAC and PHY hardware.

For ARM and PowerPC targets, VxBus and the device driver attach method obtain hardware configuration information from the DTS file.

The following is an example DTS configuration file for a network interface driver:

```
enet0: ethernet@24000 {
    #address-cells = <1>;
    #size-cells = <1>;
    cell-index = <0>;
    device_type = "network";
    model = "eTSEC";
    compatible = "motetsec";
    reg = <0x24000 0x1000>;
    ranges = <0x0 0x24000 0x1000>;
    local-mac-address = [ 00 A0 1E A0 A1 A2 ];
    interrupts = <
        29 2 0 0
        30 2 0 0
        34 2 0 0>;
    interrupt-parent = <&mpic>;
    phy-connection-type = "rgmii-id";
    phy-handle = <&phy0>;

    mdio@520 {
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "tsecMdio";
        reg = <0x520 0x20>;

        phy0: ethernet-phy@0 {
            cell-index = <0>;
            compatible = "genericPhy";
            reg = <0x0>;
        };

        phy1: ethernet-phy@1 {
            cell-index = <1>;
            compatible = "genericPhy";
            reg = <0x1>;
        };
        phy2: ethernet-phy@2 {
            cell-index = <2>;
```

```
            compatible = "genericPhy";
            reg = <0x2>;
        };

    };
};
```

Device address and IRQ information in the DTS file is retrieved by VxBus and passed to the driver through the **VXB_DEV_ID pDev** pointer. Other custom information is retrieved explicitly by the driver code (in the driver attach method).

In the attach method for your driver, the code to read the information in the DTS file is similar to the following:

```
prop = vxFdtPropGet(pFdtDev->offset, "local-mac-address", &proplen);
bcopy((char *)prop, (char*)pDrvCtrl->etsecAddr, ETHER_ADDR_LEN);
```

In the above example, the three PHY devices are children of the **enet0** device. The children are instantiated during the initialization of the **enet0** device by calling vxbDevCreate( ) and vxbDevAdd( ) for each of the PHY devices. (You driver attach method must call these functions for each PHY device.) In the example, the device driver that matches the three PHY chips is named **genericPhy**.

5. Utility Functions for MAC Drivers

There are several utility functions available to your MAC driver. These functions are provided by several libraries.

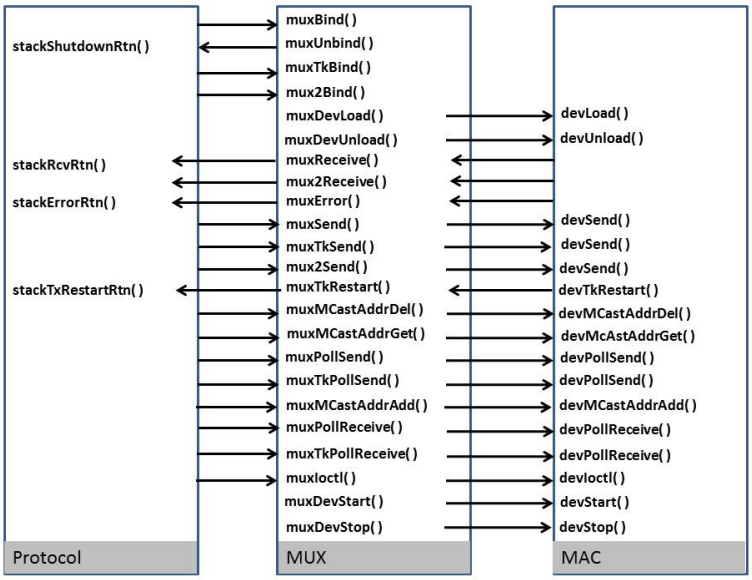
The following libraries provide utility functions for MAC drivers:

- **muxLib** - MUX interactions
- **jobQueueLib** - job queueing
- **netBufLib** - buffer management
- **vxbDmaBufLib** - DMA support

MUX Interactions

Your driver can use the functions and data structures presented in this section to interact with the MUX. As shown in the following figure, additional MUX functions are available to the network stack. However, those functions are not used by network device drivers.

Figure 2. The MUX Interface



The functions available to MAC drivers are as follows:

muxDevLoad( )

Loads a device into the MUX. Called by the MAC driver muxDevConnect( ) function.

muxDevStart( )

Starts a device already loaded into the MUX. Called by the MAC driver muxDevConnect( ) function. This function also allocates packets from the driver's pool, enables device interrupts, and enables a device for reception and transmission of packets.

muxIoctl( )

Accesses control functions.

> Note: Because it can access its own ioctl( ) function directly and does not need to go through the MUX, your network driver does not typically call muxIoctl( ).

muxDevStop( )

Stops a previously started device. This function is called from the MAC driver {vxbDevDetach}( ) method, during muxDevUnload( ), or separately. This function quiesces the device, disables interrupts, reception, and transmission. It also returns packet buffer resources that are held by the device or driver to the pool from which they came.

muxDevUnload( )

Unloads a device from the MUX. This function is called from the MAC driver {vxbDevDetach}( ) method. The best practice is to stop the device prior to calling muxDevUnload( ). However, if the device is still up, this function attempts to stop it.

muxTxRestart( )

If a device unblocks transmission after having blocked it, this function calls the stackTxRestartRtn( ) function associated with each interested protocol.

Job Queueing

VxWorks provides the **jobQueueLib** library, which network drivers can use to queue interrupt-driven work to a task context. Your driver should do a minimal amount of work in its ISR. Most work should be deferred to the task level using the **jobQueueLib** library. For more information, see Deferring ISRs with jobQueueLib on page 25.

The primary **jobQueueLib** function used by drivers is jobQueuePost( ). The prototype for this function is as follows:

```
STATUS jobQueuePost (JOB_QUEUE_ID jobQueueId, QJOB * pJob)
```

This function causes the job specified by the **pJob** argument to be executed from the context of a network processing task such as **tNet0**.

The first argument to jobQueuePost( ) is a queue ID. Your MAC driver must default to using the default queue ID, **netJobQueueId**. The parameter rxQueue00, if specified by the BSP or application, is used to find the queue in place of **netJobQueueId**. The value of the rxQueue00 parameter is a pointer to a structure of type **HEND_RX_QUEUE_PARAM**.

The jobQueueStdPost( ) function can be used instead of jobQueuePost( ) when additional flexibility is required. jobQueueStdPost( ) does not require pre-allocating a **QJOB** structure. However, when possible, use jobQueuePost( ) rather than jobQueueStdPost( ) for performance-critical code. For more information, see the reference entry for jobQueueStdPost( ).

If there is a requirement for a custom job queue, the jobQueueCreate( ) or jobQueueInit( ) functions can be used to create and initialize the custom job queue. However, the standard job queue is sufficient for most network drivers. For more information on jobQueueCreate( ) and jobQueueInit( ), see the corresponding reference entries.

A driver can use any network job queue (handled by a network task) to post its work for a device. Typically, a driver uses a single network job queue (handled by a single network task) unless it supports multiple receive or transmit queues. For performance reasons with respect to network stack operation, your driver must ensure that traffic received within a single flow is always delivered to the MUX in the same job queue.

Drivers that use only a single network job queue can allow configuration of that queue at runtime by supporting the **EIOSRCVJOBQ** ioctl( ).

Buffer Management

The functions in **netBufLib** are used to manage a pool of buffers, along with buffer-specific information contained in structures known as **M_BLK** and **C_BLK**. The **M_BLK** and **C_BLK** structures describe the packet data, and are the structures used by the network stack. The **M_BLK**, **C_BLK**, and cluster (data buffer) are known collectively as a tuple. All data transmitted from the driver to the MUX, for eventual consumption by a network stack, must be held in tuples.

A simplified interface to **netBufLib** is provided by the functions in:

installDir **/vxworks-7/pkgs_v2/net/end/lib/src/endLib.c**

This library provides functions customized to network drivers. These functions are: endPoolCreate( ), endPoolDestroy( ), endPoolTupleGet( ), and endPoolTupleFree( ).

The endPoolCreate( ) utility function is provided to create a pool suitable for use in a standard network driver. Create the pool as follows:

```
STATUS endPoolCreate
    (
    int tupleCnt,
    NET_POOL_ID * ppNetPool
    )
```

The tupleCnt argument specifies the number of tuples required. The ppNetPool argument provides a pointer to a storage location to contain the pool ID, for subsequent use by the driver. When jumbo frames are supported, endPoolJumboCreate( ) is used in place of endPoolCreate( ) in order to use 9 KB jumbo clusters (instead of the normal 1500 byte clusters).

Note: The actual cluster size is larger than the nominal MTU of 1500 or 9000 bytes. This allows for MAC headers, cache line alignment, and additional space that is useful to the stack (in some cases) for prefixing additional headers to a packet.

When your MAC driver is unloaded, it must free the pool resources by calling endPoolDestroy( ). The same function is used whether the pool is configured for standard clusters or jumbo clusters.

```
STATUS endPoolDestroy
    (
    NET_POOL_ID pNetPool
    )
```

Use endPoolTupleGet( ) and endPoolTupleFree( ) to allocate and free tuples.

```
M_BLK_ID endPoolTupleGet
    (
    NET_POOL_ID pNetPool
    )

void endPoolTupleFree
    (
    M_BLK_ID pMblk
    )
```

If the simplified interface to **netBufLib** (provided by **endLib**) does not provide sufficient flexibility, the following functions can be used as an alternative:

netTupleGet( )

Allocates a tuple from the pool.

netTupleFree( )

Returns a tuple to the pool.

netPoolCreate( )

Creates a pool of buffers to hold received packet data.

netPoolRelease( )

Releases the pool of buffers when unloading the driver.

netBufLib also includes functions to allocate individual **M_BLK**, **C_BLK**, and clusters, which the driver can then link together to form a tuple. However, for performance reasons, Wind River recommends that the driver deal only with tuples whenever possible. For more information on the additional functions, see the reference entry for **netBufLib**.

DMA Support

The functions in **vxbDmaBufLib** are used to handle address translation and cache issues required by the network device to support DMA operations. For more information on DMA drivers, see the VxWorks 7 BSP and Driver Guide.

PHY and MII Bus Interactions on page 16 Several functions are available to you to support MII bus management from the MAC driver. These are required for managing PHY devices in the system.

5.1. PHY and MII Bus Interactions

Several functions are available to you to support MII bus management from the MAC driver. These are required for managing PHY devices in the system.

When the network interface (MAC) drivers initialize the network link, they must determine what PHY devices are present and they must configure those PHY devices. In your driver, you can do this by creating a subordinate MII bus, and then creating a list of PHY devices on that MII bus. You can then determine which PHY devices are present and which of the devices are connected to the network. The following functions are available to help with MII bus management:

- miiBusCreate( )
- miiBusDelete( )
- miiBusModeGet( )
- miiBusModeSet( )
- miiBusMediaListGet( )

For more information, see the reference documentation for **miiBus**.

miiBusCreate( )

The miiBusCreate( ) function is defined as follows:

```
STATUS miiBusCreate
        (
        VXB_DEV_ID pBus,
        UINT32 flags
        )
```

The miiBusCreate( ) function is used to instantiate PHY device instances and attach them to a parent MDIO bus. The parent can be any VxBus node that implements either on or both of the sets of {mdioRead}( ) and {mdioWrite}( ), and {mdio45Read}( ) and {mdio45Write}( ), methods. Usually this is an MDIO device. However it may also be an Ethernet MAC device that has a direct parent of a child PHY device. The miiBusCreate( ) function will automatically probe PHYs by performing an MII read of the status register at every PHY address from 0 to 31. Any linkages between PHYs and MACs that are specified in the device tree file in the BSP will be honored. The flags argument specifies any special properties for the MDIO bus. Currently only two flags are supported: **VXB_MDIO_FLAG_CLAUSE22** and **VXB_MDIO_FLAG_CLAUSE45**. These flags are mutually exclusive. A single MDIO parent should not have both clause 22 and clause 45 devices present on the same 2-wire management bus. If an MDIO instance has clause 22 PHYs, it should use the **VXB_MDIO_FLAG_CLAUSE22** flag. For MDIO instances that manage clause 45 devices, it should use **VXB_MDIO_FLAG_CLAUSE45**. Note that it is possible for one driver to support both kinds of devices, but not at the same time. (For example, the MEMAC MDIO driver used with T2080 and T4240 processors can select either clause 22 or clause 45 mode depending on how the system is configured, but each MDIO instance is configured for one more or the other at start-up, and can not switch modes on the fly.)

When a PHY node is added, the MAC parent of the PHY will have its {miiMediaAttach}( ) method called to notify it of the new device's existence.

miiBusDelete()

The miiBusDelete( ) function is defined as follows:

```
STATUS miiBusDelete
        (
        VXB_DEV_ID pBus
        )
```

This function is used by an MDIO parent to delete all the child PHY devices that it manages. The underlying device nodes will be removed from the VxBus tree. If the parent MAC of a given PHY supports the {miiMediaDetach}( ) method, it will be called so that it knows the PHY device is being removed. (This handles the situation where the MAC parent and MDIO parent of a given PHY are not the same device.)

miiBusModeGet( )

The miiBusModeGet( ) function is defined as follows:

```
STATUS miiBusModeGet
    (
    VXB_DEV_ID pDev,
    UINT32 * pMode,
    UINT32 * pSts
    )
```

This function is provided as an interface to the {miiModeGet}( ) methods exported by the individual PHY drivers. The **pMode** and **pSts** arguments are specified in exactly the same manner as the arguments to func{miiModeGet}( ) described in Device Driver Methods for PHY Drivers on page 48. This function is used by Ethernet MAC drivers to query the current link state and characteristics. This in turn leads to calls to the MAC driver's {miiRead}( ) method to access the PHY.

miiBusModeSet( )

The miiBusModeSet( ) function is defined as follows:

```
STATUS miiBusModeSet
    (
    VXB_DEV_ID pDev,
    UINT32 mode
    )
```

Like miiBusModeGet( ), this function provides an interface to the {miiModeSet}( ) methods exported by individual PHY drivers. This function is typically called from a MAC driver's start function in order to initialize the network link to a known state (typically **autoneg**). This function can be called at any time to change the link state to any desired setting.

Note: In order to reduce the number of register accesses performed, call miiBusModeGet( ) and miiBusModeSet( ) as infrequently as possible. For example, your MAC driver can call miiBusModeGet( ) only when its {miiMediaUpdate}( ) method is invoked and then cache the results, rather than calling miiBusModeGet( ) every time its **EIOCGIFMEDIA** ioctl( ) handler is called.

miiBusMediaListGet( )

The miiBusMediaListGet( ) function is defined as follows:

```
STATUS miiBusMediaListGet
    (
    VXB_DEV_ID pDev,
```

```
    END_MEDIALIST ** ppMediaList
    )
```

This function returns a pointer to an **END_MEDIALIST** structure that contains entries for all of the media supported by the PHYs on the specified MII bus. Your MAC driver can use this information when providing responses to **EIOCGIFMEDIALIST** ioctl( ) queries. This structure also includes a default media setting that specifies the default media type for this bus. Typically, the default value is **IFM_AUTO**.

MII Methods for MAC Driver

In addition, there are several MII methods that a MAC driver may need to implement. Note that the pDev handle arguments in these methods refer to the handle for the parent MAC device, not for a PHY.

The {miiRead}( ) and {miiWrite}( ) methods must be implemented if the MAC also provides MDIO access. This is usually the case for standalone plug-in network adapter cards as they usually either have a PHY directly integrated with the MAC, or connected to some of the MAC's I/O pins. By contrast, many SoC processors include one or more MACs, but have external PHYs which are managed with I/O pins whose registers are not part of the MAC register block and are hence considered separate devices. (They might be managed by a different VxBus node than the MAC.)

```
LOCAL STATUS miiRead
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT16 regAddr,
    UINT16 * dataVal
    )
    {
    return (OK);
    }
```

```
LOCAL STATUS miiWrite
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT16 regAddr,
    UINT16 dataVal
    )
    {
    return (OK);
    }
```

The {mii45Read}( ) and {mii45Write}( ) methods provide similar functionality to the {miiRead}( ) and {miiWrite}( ) methods, except that they use the clause 45 MDIO protocol. These routines may be needed for 10 gigabit ethernet or faster devices. They accept an extra devAddr argument which is used to specify which sub-device to access when performing a read or write transaction.

```
LOCAL STATUS mii45Read
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 devAddr,
    UINT16 regAddr,
    UINT16 * dataVal
    )
    {
```

```
    return (OK);
    }
```

```
LOCAL STATUS mii45Write
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 devAddr,
    UINT16 regAddr,
    UINT16 dataVal
    )
    {
    return (OK);
    }
```

The {miiMediaUpdate}( ) method is used as a callback by the miiBusMonitor task to notify MAC drivers when a link state change has occurred. This usually reflects network cable unplug and replug events. In some cases it may also be called when miiBusModeSet( ) is invoked to change the link settings, as this may trigger a PHY reset. The MAC driver may call miiBusModeGet( ) to obtain the current media settings for its child PHY handle(s) to determine what state changes have occured.

```
LOCAL STATUS miiMediaUpdate
    (
    VXB_DEV_ID pDev
    )
    {
    return (OK);
    }
```

The miiMediaAttach}( ) method is used as a callback by the MII library to notify a MAC that its PHY has been discovered and attached. The pPhy argument is a PHY device handle which the parent can save for use in the {miiMediaUpdate}( ) method or other link management code (for example, the IFMEDIA ioctl handlers). The linkage between a MAC and PHY may be decided in one of the following ways:

- if there's a PHY-handle linkage for the MAC specified in the BSP's device tree source (**.dts**) file
- if the MAC driver with its own {miiRead}( ) and{miiWrite}( ) methods called miiBusCreate( ) and child PHY devices were discovered

The former case typically occurs with MAC devices integrated into SoCs, while the latter typically occurs with standalone plug-in network adapters such as PCI Ethernet cards.

```
LOCAL STATUS miiMediaAttach
    (
    VXB_DEV_ID pDev,
    VXB_DEV_ID pPhy
    )
    {
    return (OK);
    }
```

The {miiMediaDetach}( ) is used as the complementary callback to {miiMediaAttach}( ). It will be invoked when a PHY device is deleted from the VxBus tree, usually as the result of a call to miiBusDelete( ). Once the MAC driver receives this callback, it must cease all link-management attempts using the specified PHY handle since it will no longer be valid.

```
LOCAL STATUS miiMediaDetach
    (
```

```
    VXB_DEV_ID pDev,
    VXB_DEV_ID pPhy
    )
    {
    return (OK);
    }
```

Parent topic: Utility Functions for MAC Drivers on page 12

6. MAC Driver Initialization

The MAC driver is initialized during the VxBus initialization phase.

The entire initialization of the MAC device driver is performed in the context of the MAC driver's VxBus {devAttach}( ) method (see VxBus Device Driver Methods for MAC Drivers on page 5). This method is responsible for retrieving configuration information for the MAC device from the device tree binary (DTB), allocating resources (memory access and interrupt configuration), hardware initialization, DMA setup (if applicable), calls to PHY device initialization, and, finally, a call to muxDevConnect( ).

The call to the {devAttach}( ) method is delayed for network drivers until the relevant MUX support is available.

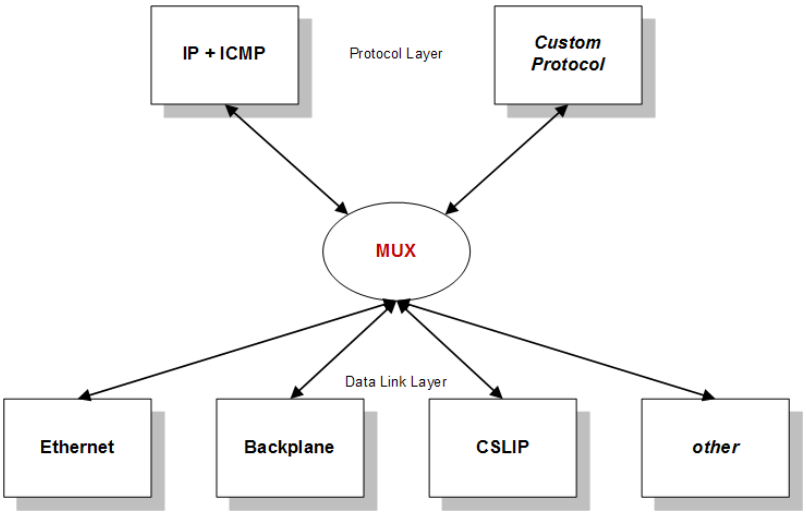
The MUX completes the **END_OBJ** structure by setting various structure members, including a function pointer that the driver receive handler calls to deliver received packets to the MUX. The MUX then adds the returned **END_OBJ** to a linked list of **END_OBJ** structures. This list maintains the state of all currently active network devices on the system. After control returns from muxDevLoad( ), your driver is loaded and ready to use.

7. MUX Interface

The multiplexor (MUX) is used to connect your networking interface device driver to the VxWorks networking code.

The MUX is an interface that joins the data link and protocol layers. A MAC driver does not interface directly with network layer protocols. Rather, it interfaces with the MUX, which is an abstraction layer that de-couples the driver from any particular protocol. The MUX API multiplexes access to the networking hardware for multiple network protocols. The following figure shows the MUX in relationship to the protocol and data link layers:

Figure 3. MUX Interface Between the Protocol and Data Link Layers



Note: The data link layer is an abstraction. A network interface driver is code that implements the functionality described by that abstraction. Likewise, the protocol layer is an abstraction. The code that implements the functionality of the protocol layer can be considered to be a protocol driver. However, this document refers to such code simply as the *protocol*.

MAC (END) drivers must include the header **endLib.h** in order to obtain most of the declarations needed to interface with the MUX. The **endLib.h** header in turn includes the **end.h** and **muxLib.h** files.

The **END_OBJ** type defined in the **end.h** header is the type that represents an END device (that is, a network interface) to the MUX. The MAC driver control structure for a device instance must include an **END_OBJ** structure. Conventionally, the **END_OBJ** structure is the first member of the MAC driver device control structure.

Typically, the driver only directly accesses a few members of the **END_OBJ** structure. Other members are there for the benefit of the MUX and should be considered semi-private or hidden from the point of view of the MAC driver.

END_OBJ includes a pointer to a **NET_FUNCS** table. This table contains function pointers to the functions that an END driver must (in most cases) provide for the MUX. The driver typically defines a single static **NET_FUNCS** structure, shared by all its devices. The function pointers in the structure point to either driver-provided or generic functions.

```
typedef struct net_funcs
    {
    STATUS (*start) (END_OBJ*);                 /* Driver's start func. */
    STATUS (*stop) (END_OBJ*);                  /* Driver's stop func. */
    STATUS (*unload) (END_OBJ*);                   /* Driver's unload func. */
    int (*ioctl) (END_OBJ*, int, caddr_t);    /* Driver's ioctl func. */
    STATUS (*send) (END_OBJ* , M_BLK_ID);          /* Driver's send func. */
    STATUS (*mCastAddrAdd) (END_OBJ*, char*); /* Driver's mcast add func. */
    STATUS (*mCastAddrDel) (END_OBJ*, char*); /* Driver's mcast delete func. */
    STATUS (*mCastAddrGet) (END_OBJ*, MULTI_TABLE*);
                                            /* Driver's mcast get fun. */
    STATUS (*pollSend) (END_OBJ*, M_BLK_ID);  /* Driver's polling send func. */
    STATUS (*pollRcv) (END_OBJ*, M_BLK_ID);   /* Driver's polling recv func. */

    /*
     * The minimum required arguments for (*formAddress)() are:
     * (*formAddress)(M_BLK_ID, M_BLK_ID, M_BLK_ID, BOOL)
     */
    M_BLK_ID (*formAddress) ();         /* Driver's addr formation func. */
```

```
    /*
     * The minimum required arguments for (*packetDataGet)() are
     * (*packetDataGet)(M_BLK_ID, LL_HDR_INFO *)
     */
    STATUS (*packetDataGet) ();         /* Driver's packet data get func. */
    STATUS (*addrGet) (M_BLK_ID, M_BLK_ID, M_BLK_ID, M_BLK_ID, M_BLK_ID);
                                          /* Driver's packet addr get func. */
    int (*endBind) (void*, void*, void*, long type);
                                        /* information exchange between */
                                        /* network service and network driver */
    } NET_FUNCS;
```

The endBind( ) function is typically not used, and is left NULL.

For an Ethernet driver, the formAddress, packetDataGet, and addrGet function pointers are typically set to the functions endEtherAddressForm( ) (or end8023AddressForm( )), endEtherPacketDataGet( ), and endEtherPacketAddrGet( ), respectively. The **endEtherHdr** library provides these functions, which construct or parse Ethernet and 802.3 headers. A driver for a device using non-Ethernet headers must provide its own version of these functions.

The remaining functions must generally be provided by the driver.

Your MAC driver causes the device instance that it controls to be loaded into the MUX as an END device by calling muxDevLoad( ):

```
IMPORT DEV_COOKIE muxDevLoad
    (
    int unit,
    END_OBJ* (*endLoad) (char *, void*),
    char *initString,
    BOOL loaning,
    void*pBSP
    )
```

In the above example, the unit argument is the END unit number for the device, which is taken to be the same as the VxBus unit number of the device instance.

The most important argument to muxDevLoad( ) is the driver **endLoad** function, which muxDevLoad( ) calls twice in the course of loading the device. Your MAC driver must provide this function.

The first time that muxDevLoad( ) calls the endLoad( ) function, it passes a pointer to a buffer of length END_NAME_MAX containing the empty string as the first argument and NULL as the second argument. In response, the driver endLoad( ) function copies the driver name (for example, **gei** for the gei driver) into the buffer pointed to by the first argument, and returns NULL.

The second time that muxDevLoad( ) calls the endLoad( ) function, it passes a pointer to a buffer of length END_INIT_STR_MAX as the first argument. The second argument is the **pBSP** pointer passed to muxDevLoad( ). (VxBus drivers typically pass the VXB_DEV_ID for the device instance in the pBSP argument.)

The buffer passed in the first endLoad( ) argument contains the decimal representation of the unit number, followed by a colon, **:**. This is followed by the string passed in the initString argument. The driver can ignore this information if it does not need it. If all of this does not fit, muxDevLoad( ) returns an error, NULL-terminated, in END_INIT_STRING_MAX bytes of the buffer.

The second time that the endLoad( ) function is called, it returns a pointer to the partially initialized **END_OBJ** structure that the driver provides for the device instance, or it returns NULL if a failure occurs.

An example endLoad( ) function showing the typical behavior is the fslImxEnetEndLoad( ) function.

```
LOCAL END_OBJ *fslImxEnetEndLoad
    (
    char * loadStr,
    void * pArg
    )
    {
    ENET_DRV_CTRL *pDrvCtrl;
    VXB_DEV_ID pDev;

    /* make the MUX happy */

    if (loadStr == NULL)
    return (NULL);

    if (loadStr[0] == 0)
        {
        bcopy (ENET_NAME, loadStr, sizeof(ENET_NAME));
        return (NULL);
        }

    pDev = pArg;
    pDrvCtrl = (ENET_DRV_CTRL *)vxbDevSoftcGet (pDev);

    if (END_OBJ_INIT (&pDrvCtrl->fecEndObj, NULL, ENET_NAME, pDrvCtrl->unit,
                      &fslImxEnetNetFuncs,
                      "Freesacle i.MX ENET FDT END driver") == ERROR)
        {
        ENET_DBG (ENET_DBG_ERR, "%s%d: END_OBJ_INIT failed\n",
                      (_Vx_usr_arg_t)ENET_NAME,
                      pDrvCtrl->unit, 0, 0, 0, 0);
        return (NULL);
        }

    if (endM2Init (&pDrvCtrl->fecEndObj, M2_ifType_ethernet_csmacd,
                   pDrvCtrl->fecAddr, ETHER_ADDR_LEN, ETHERMTU, 100000000,
                   IFF_NOTRAILERS | IFF_SIMPLEX | IFF_MULTICAST |
                   IFF_BROADCAST) == ERROR)
        {
        ENET_DBG (ENET_DBG_ERR, "%s%d: endM2Init failed\n",
                  (_Vx_usr_arg_t)ENET_NAME,
                  pDrvCtrl->unit, 0, 0, 0, 0);
        return (NULL);
        }

    /* allocate a buffer pool */

    pDrvCtrl->fecMaxMtu = ENET_MTU;
    if (endPoolCreate (ENET_TUPLE_CNT, &pDrvCtrl->fecEndObj.pNetPool) == ERROR)
        {
        ENET_DBG (ENET_DBG_ERR, "%s%d: pool creation failed\n",
                  (_Vx_usr_arg_t)ENET_NAME,
                  pDrvCtrl->unit, 0, 0, 0, 0);
        return (NULL);
        }

    pDrvCtrl->fecPollBuf = endPoolTupleGet (pDrvCtrl->fecEndObj.pNetPool);
```

```
    /* set up polling stats */

    pDrvCtrl->fecEndStatsConf.ifPollInterval = sysClkRateGet();
    pDrvCtrl->fecEndStatsConf.ifEndObj = &pDrvCtrl->fecEndObj;
    pDrvCtrl->fecEndStatsConf.ifWatchdog = NULL;
    pDrvCtrl->fecEndStatsConf.ifValidCounters = (END_IFINUCASTPKTS_VALID |
            END_IFINMULTICASTPKTS_VALID | END_IFINBROADCASTPKTS_VALID |
            END_IFINOCTETS_VALID | END_IFINERRORS_VALID | END_IFINDISCARDS_VALID |
            END_IFOUTUCASTPKTS_VALID | END_IFOUTMULTICASTPKTS_VALID |
            END_IFOUTBROADCASTPKTS_VALID | END_IFOUTOCTETS_VALID |
            END_IFOUTERRORS_VALID);

    /* set up capabilities */

    pDrvCtrl->fecCaps.cap_available = IFCAP_VLAN_MTU;
    pDrvCtrl->fecCaps.cap_enabled = IFCAP_VLAN_MTU;

    return (&pDrvCtrl->fecEndObj);
    }
```

Assuming there are no failures in either the driver's endLoad( ) function or in muxDevLoad( ), the END device is registered in the MUX and becomes accessible by protocols.

After calling muxDevLoad( ) successfully, the driver's attach function then calls muxDevStart( ) to start the END device. This results in a call to the driver's start function, provided in the **NET_FUNCS** structure. The start function does what is necessary to activate the device so that reception and transmission can occur.

After calling muxDevStart( ), the driver calls endPollStatsInit( ) to enable polled-mode collection of device statistics. The calls to muxDevLoad( ), muxDevStart( ), and endPollStatsInit( ) are often collected together into a separate function as shown in the following example from the tei driver:

```
LOCAL void teiMuxConnect
    (
    VXB_DEV_ID pDev
    )
    {
    TEI_DRV_CTRL * pDrvCtrl;

    pDrvCtrl = vxbDevSoftcGet (pDev);

    /* Save the cookie. */

    pDrvCtrl->teiMuxDevCookie = muxDevLoad (pDrvCtrl->teiUnit,
        teiEndLoad, "", TRUE, pDev);

    if (pDrvCtrl->teiMuxDevCookie != NULL)
        (void) muxDevStart (pDrvCtrl->teiMuxDevCookie);

    if (_func_m2PollStatsIfPoll != NULL)
        (void) endPollStatsInit (pDrvCtrl->teiMuxDevCookie,
            _func_m2PollStatsIfPoll);

    return;
    }
```

8. Deferring ISRs with jobQueueLib

When working with VxWorks network drivers, you must understand why network drivers defer their ISR processing to task context, and how they accomplish this.

Many desktop and mainframe operating systems use network drivers that perform a significant amount of work at the interrupt level. The device ISR in these drivers typically drains any packets in the receive ring, at least queueing them for the stack, but sometimes also executing a fair amount of network stack code. The ISR also typically frees any completed packets in the transmit ring.

Because VxWorks is intended for real-time applications, ISRs must be kept short. Wind River does not recommend the use of long ISRs for network packet processing. For this reason, most of the network stack processing for incoming packets and completed transmits that would (in some operating systems) be done from within an ISR, is pushed to a task context in VxWorks. This is accomplished using **jobQueueLib**.

Interrupt Handlers

Upon arrival of an interrupt on a network device, VxWorks invokes the driver's previously registered ISR. Your driver ISR must do the minimum amount of work necessary to acknowledge the event causing the interrupt, and when appropriate, disable further device interrupts of the same nature. To minimize the time that the current interrupt delays task-level execution (or servicing of other, lower priority interrupts), the ISR must handle only those tasks that require minimal execution time. The ISR must queue all time-consuming work for processing at the task level.

Aside from the general practice of limiting the amount of work done in an ISR, in VxWorks, it is not possible to directly call the MUX receive entry point from an ISR. Instead, it must be called from a task context. To queue packet-reception work for processing at the task level, your ISR must call jobQueuePost( ). (For information on jobQueuePost( ), see Utility Functions for MAC Drivers on page 12.)

9. Transferring Data with M_BLKs

Network interface drivers pass and receive packet data to and from the MUX through structures called M_BLKs. The API in the netBufLib( ) library provides a means of managing these M_BLK structures and their data.

Each network interface requires its own memory pool for data and for M_BLK structures. Received data is put in the data blocks allocated from this pool, and sent to the MUX. Data for transmission is allocated from the system pool. Once the data are sent, the driver must free data blocks and M_BLK structures, so that they can be returned to the system pool.

Note: Network protocols can pass M_BLK chains in which some, but not all, M-BLKs have a zero data length. If your hardware device cannot handle zero-length data segments, your driver must filter out the zero-length M_BLKs and only pass data segments from M_BLKs that have a nonzero length to the device.

Setting Up a Memory Pool for M_BLK-Style Drivers

Each MAC driver unit requires its own memory pool. Network interface drivers typically use a pool with a single fixed block size, so that a single cluster is large enough to hold an entire received packet with little or no wasted space. (The term *cluster* is used to refer to buffers containing packet data.) A typical Ethernet driver cluster size is 1500 bytes for the maximum transmission unit (MTU), plus sufficient extra space to accommodate the MAC header and to ensure cache line alignment of the cluster boundaries. In some cases, an additional one to two cache lines are provided for various reasons. (For more information on memory pools, see the reference entry for **netBufLib**.)

The following code is a simplified version of the code in the **vxbFdtFslImxEnd** driver used to create a driver pool.

You should make the pool at least several times as big as the RX DMA ring, a commonly used multiple is 3. Extra packets are often added to support coalescing in the TX path. However, Wind River recommends that you provide a per-device configuration mechanism that allows the user to increase the size of the pool without rebuilding the driver.

```
/* allocate a buffer pool */

    pDrvCtrl->fecMaxMtu = ENET_MTU;
    if (endPoolCreate (ENET_TUPLE_CNT, &pDrvCtrl->fecEndObj.pNetPool) == ERROR)
        {
        ENET_DBG (ENET_DBG_ERR, "%s%d: pool creation failed\n",
                  (_Vx_usr_arg_t)ENET_NAME,
                  pDrvCtrl->unit, 0, 0, 0);
        return (NULL);
        }
```

Once the pool is created, the driver allocates tuples with a call to endPoolTupleGet( ).

```
pDrvCtrl->fecPollBuf = endPoolTupleGet (pDrvCtrl->fecEndObj.pNetPool);
```

Supporting Scatter-Gather with M_BLK-Style Drivers

Scatter-gather is a direct memory access (DMA) technique that allows for a single large block of data to be distributed among multiple clusters. When data is being sent, the DMA engine within the device gathers data from each cluster in turn, and sends it out to the output stream. When data is being received, the DMA engine within the device scatters data into multiple clusters, filling each cluster in turn. For performance reasons, if a device supports scatter-gather, the driver for the device must support scatter-gather as well, at least for output.

When transmitting packets, the network stack is often unable to find a single cluster that is large enough to hold a large packet. When this happens, it obtains multiple tuples with clusters that have sufficient total space to hold the packet and links them together to form an M_BLK chain. The network stack copies the data into the chained clusters, then sends the fragmented packet to the driver as an M_BLK chain.

When scatter-gather is not supported and the network stack sends a fragmented packet to the driver, the driver must coalesce the M_BLK chain. This involves allocating a cluster from the driver pool and copying the packet fragments from the M_BLK chain into the newly allocated cluster. Because VxWorks network drivers typically create pools with clusters large enough to hold an entire packet, the fragmented data fits within a single cluster. However, this copy operation is time consuming and results in lower network throughput.

When a device supports scatter-gather for transmit, it can continue DMA across multiple fragments by following a list of fragment buffer pointer and size pairs. A driver written for such a device walks the M_BLK chain, extracts the cluster buffer pointers and the fragment sizes, and then forms a gather list according to the device's specification. This is typically faster than the copy operation required when the M_BLK chain is coalesced.

Each fragment, or link of the M_BLK chain, requires a DMA descriptor or a portion of a DMA descriptor, depending on the design of the device. If there are not enough free descriptors to hold the M_BLK chain, or if there are not enough scatter-gather slots in the descriptor, then the M_BLK chain cannot be sent using scatter-gather, but must be coalesced into a single cluster. The driver send function is responsible for determining whether the M_BLK chain can be sent using scatter-gather, or whether it needs to be coalesced. To determine whether or not there are sufficient resources available to hold the packet data, a send function must count the number of fragments in the M_BLK chain, and compare that number with the number of available descriptors or the number of scatter-gather slots available in a descriptor. If the M_BLK chain cannot be sent using scatter-gather, the driver must either coalesce the M_BLK chain into a single cluster allocated from the driver's pool, or it must discard the packet.

In most VxWorks network drivers, scatter-gather is not a concern for packet reception. This is because the driver's buffers are all of a single size and are sufficient to hold the maximum incoming frame (MTU). Therefore, VxWorks network drivers do not fragment incoming frames.

Note: You can also use jobQueuePost( ) to queue up work other than processing of received packets.

CAUTION: **The jobQueuePost( ) function requires that you provide a pre-allocated QJOB structure describing the job being posted. This QJOB is typically embedded in the driver control structure for the device instance. A given enqueued QJOB cannot be reused until it has been taken off the job queue to be serviced; if the same QJOB is posted again to a job queue while it is currently enqueued, the second jobQueuePost( ) call fails with no effect.**

10. Networking Protocol Restrictions and Optimizations

Although the MUX enforces a distinction between the driver and the protocol, there are a few restrictions that the protocol imposes on drivers and a few protocol-specific optimizations available to drivers to use where appropriate.

Checksum Offload for Internet Family Protocols on page 27TCP/IP checksum offload eliminates host-side checksum calculation and verification overhead by performing checksum computation with hardware assist. Many devices provide support for this feature.
Driver Checksum Offloading and Receiving on page 32 Your driver's receive function is responsible for certain tasks related to checksum offloading on the receiving side.
Corrupt Packet Handling on page 33 You can use your device hardware to indicate that a transport checksum is good. However, your driver must verify any device-reported invalid checksums.

10.1. Checksum Offload for Internet Family Protocols

TCP/IP checksum offload eliminates host-side checksum calculation and verification overhead by performing checksum computation with hardware assist. Many devices provide support for this feature.

The Wind River network stack and network drivers can jointly support offloading transport-layer checksum calculations for TCP and UDP over both IPv4 and IPv6. The network stack supports checksum offload for received packets only. (That is, the network stack never transmits a packet requesting TX checksum offload from the driver.) Offload of VLAN tag insertion and removal is also supported.

Note: The Wind River Network Stack does not attempt to support offload of IPv4 header checksum, and provides no interface to request it or report it on a per-packet basis.

The programming interface for checksum offload between the network stack and network devices consists of two parts:

- A network driver ioctl( )-based interface that allows a driver to report the offload capabilities of the device it manages, and allows the stack to, at a coarse level, administratively enable or disable certain of those capabilities.
- A per-packet interface for the stack to request checksum offload on packets to be transmitted, or for the driver to report checksum offload results on received packets.

The network driver ioctl( )-based interface for reporting or managing offload capabilities is mostly the same between traditional M_BLK-oriented MAC drivers; but the per-packet offload interface is significantly different.

A network driver that supports checksum offload (or certain VLAN- related capabilities) must implement the **EIOCGIFCAP** and **EIOCSIFCAP** ioctl( ) commands. The first is called by the stack to get the (offload-related) capabilities of the interface, while the second can be called to administratively enable or disable particular capabilities. Both of these ioctl( ) functions provide a pointer to an **END_CAPABILITIES** structure as the ioctl( ) argument. This structure is defined in:

installDir **/vxworks-7/pkgs_v2/net/end/lib/h/endCommon.h**

The structure is defined as follows:

```
typedef struct _END_CAPABILITIES
    {
    uint32_t cap_available; /* supported capabilities (RO) */
    uint32_t cap_enabled;   /* subset of above which are enabled (RW) */
    uint32_t csum_flags_tx; /* cap_enabled mapped to CSUM flags
                               for TX (RO) */
    uint32_t csum_flags_rx; /* cap_enabled mapped to CSUM flags
                               for RX (RO) */
    } END_CAPABILITIES;
```

For the EIOCGIFCAP command, the driver must fill in all fields of the structure. It sets **cap_available** to a bit mask with bits on indicating the capabilities it supports, taken from the following:

IFCAP_RXCSUM

The device can perform some checksum offload on received packets.

IFCAP_TXCSUM

The device can perform some checksum offload on transmitted packets.

IFCAP_JUMBO_MTU

The device supports an MTU of 9000 bytes.

IFCAP_VLAN_MTU

The device supports a VLAN-compatible MTU. That is, the addition of a 4 byte VLAN tag to the MAC header does not require decreasing the MTU by 4 bytes (for example, from 1500 to 1496).

IFCAP_VLAN_HWTAGGING

The device supports insertion of VLAN tags per-packet on transmission in hardware, or supports extraction of VLAN tags from received packets.

In addition, the following flags are defined in the interface, but are not fully supported at present by the Wind River Network Stack:

IFCAP_TCPSEG

The device supports TCP segmentation of "large TCP sends" in hardware.

IFCAP_IPSEC

The device supports IPSEC cryptographic offload.

IFCAP_NETCONS

The device can act as a network console.

IFCAP_IPCOMP

The device can do IPcomp.

IFCAP_CAP0

Vendor-specific capability 0.

IFCAP_CAP1

Vendor-specific capability 1.

IFCAP_CAP2

Vendor-specific capability 2.

IFCAP_CAP3

Vendor-specific capability 3.

The **cap_enabled** member is set with the currently enabled subset of the available capabilities. At load time, the convention is that the driver sets **cap_enabled** equal to **cap_available**. However, the stack may subsequently change **cap_enabled** using the **EIOCSIFCAP** ioctl( ) function.

The **csum_flags_tx** and **csum_flags_rx** members give a more detailed description of the driver's capabilities, using a different set of flags. The **csum_flags_tx** member is important because it governs what sorts of per-packet offload requests the stack can make of the driver on transmit. The **csum_flags_rx** member is currently only informational. The **csum_flags_tx** and **csum_flags_rx** members use the same set of flags that are used as M_BLK flags as part of the per-packet checksum offload interface for M_BLK-style drivers. However, they are still used as part of the capabilities interface.

The flags are defined in:

installDir **/vxworks-7/pkgs_v2/net/net_base/h/net/mbuf.h**

The following table provides the descriptions for these flags.

Table 1. Hardware and Software Checksum Support Flags

Flag	Transmit/Receive	Description
CSUM_TCPv6	TX	The device can calculate and insert the IPv4 header checksum.
	RX	The device can verify the IPv4 header checksum.
CSUM_TCP	TX	The device can calculate and insert the TCP checksum in a TCP packet over IPv4.
	RX	The device can verify or calculate the TCP checksum of a TCP packet over IPv4
CSUM_UDP	TX	The device can calculate and insert the UDP checksum in a UDP packet over IPv4.
	RX	The device can verify or calculate the UDP checksum of a UDP packet over IPv4.
CSUM_TCPv6	TX	The device can calculate and insert the TCP checksum in a TCP packet over IPv6.
	RX	The device can verify or calculate the TCP checksum of a TCP packet over IPv6.
CSUM_UDPv6	TX	The device can calculate and insert the UDP checksum in a UDP packet over IPv6.
	RX	The device can verify or calculate the UDP checksum of a UDP packet over IPv6.

Flag	Transmit/Receive	Description
CSUM_VLAN	TX	The device can insert an arbitrary VLAN tag into the Ethernet header of an outgoing frame, on a per-packet basis.
	RX	The device can extract and remove the VLAN tag from a received packet and report it to the stack.
CSUM_PTAGGED	TX	The device can send a priority-tagged frame. That is, a frame with VLAN tag with VLAN ID zero.
	RX	The device can report reception of priority tagged frames.

Note: The stack does not make use of IPv4 header checksum offload.

For VxWorks network drivers, the following capabilities flags are defined but are not supported by the network stack:

CSUM_IP_FRAGS (TX)

The device can perform checksum across IPv4 fragments.

CSUM_FRAGMENT (TX)

The device can perform IPv4 fragmentation of frames too large for the link MTU.

CSUM_TCP_SEG (TX)

The device can perform TCP segmentation of large TCP/IPv4 sends.

CSUM_TCPv6_SEG (TX)

The device can perform TCP segmentation of large TCP/IPv6 sends.

The following flags are not used as capabilities, but rather are used only in the per-packet offload interface, in the M_BLK **mFlags** member.

CSUM_IP_CHECKED (RX)

The IPv4 header checksum for this packet was checked.

CSUM_IP_VALID (RX)

The IPv4 header checksum for this packet was found to be valid. Used only in combination with **CSUM_IP_CHECKED**.

CSUM_DATA_VALID (RX)

The **csum_data** field in the **M_PKT_HDR** structure is valid. Contents depend on whether or not **CSUM_PSEUDO_HDR** is also set.

CSUM_PSEUDO_HDR (RX)

If set, the **csum_data** field contains the un-complemented ones' complement sum over the TCP or UDP header, transport payload, and the IP pseudo-header used in calculating the TCP or UDP checksum. If not set, the sum in the **csum_data** field does not include the IP pseudo-header sum. This flag is significant only in conjunction with **CSUM_DATA_VALID**.

The following are combinations or aliases of various previously listed flags:

- **CSUM_DELAY_DATA** is (**CSUM_TCP** | **CSUM_UDP**).
- **CSUM_DELAY_IP** is (**CSUM_IP**).
- **CSUM_DELAY_DATA6** is (**CSUM_TCPv6** | **CSUM_UDPv6**).
- **CSUM_RESULTS** is (**CSUM_IP_CHECKED** | **CSUM_IP_VALID** | **CSUM_DATA_VALID** | **CSUM_PSEUDO_HDR**).

The following macros are used to extract transmit checksum-offload related information from a packet:

CSUM_IPHDR_OFFSET

Extracts the byte offset of the start of the IP header from the start of the packet.

```
#define CSUM_IPHDR_OFFSET(pMblk) ((pMblk)->mBlkHdr.offset1)
```

CSUM_IP_HDRLEN

Extracts the length in bytes of the IP header. For IPv6, this includes any extension headers before the transport header.

```
#define CSUM_IP_HDRLEN(pMblk) ((pMblk)->mBlkHdr.offset2)
```

CSUM_XPORT_HDRLEN

Extracts the length in bytes of the transport-layer header. Note that this macro is not supported by the network stack.

```
#define CSUM_XPORT_HDRLEN(pMblk) \
(((pMblk)->mBlkPktHdr.csum_data & 0xff00) >> 8)
```

CSUM_XPORT_CSUM_OFF

Extracts the byte offset of the transport layer checksum field relative to the start of the transport header.

```
#define CSUM_XPORT_CSUM_OFF(pMblk) ((pMblk)->mBlkPktHdr,csum_data & 0xff)
```

The network stack (or a management application) can use the **EIOCSIFCAP** ioctl( ) to change which offload capabilities of the driver are enabled. The caller sets the **cap_enabled** member of the **END_CAPABILITIES** structure specified to the desired set of capabilities. The driver ioctl( ) code sets its current capabilities accordingly, ignoring any capabilities that the caller specifies but the driver does not support. Although the driver can modify the device hardware settings in response to a change in enabled capabilities, it does not typically do so. For instance, if the device supports receive checksum offload, but a management utility calls the **EIOCSIFCAP** ioctl( ) to disable **IFCAP_RXCSUM**, current drivers can discard the checksum offload information reported by the device for received datagrams, rather than passing it on to the stack; receive checksum offload is still enabled in the hardware.

A network driver's **EIOCGIFCAP** and **EIOCSIFCAP** ioctl( ) handlers are usually simple, as in this fragment of a switch statement from the **vxbFdtFslImxEnd.c** driver's fslImxEnetEndIoctl( ) function:

```
        case EIOCGIFCAP:
             hwCaps = (END_CAPABILITIES *)data;
             if (hwCaps == NULL)
                 {
                 error = EINVAL;
                 break;
                 }
             hwCaps->cap_available = pDrvCtrl->fecCaps.cap_available;
             hwCaps->cap_enabled = pDrvCtrl->fecCaps.cap_enabled;

             break;
```


WNDRVR

```
        /*
         * The only special capability we support is VLAN_MTU, and
         * it can never be turned off.
         */

        case EIOCSIFCAP:
            error = ENOTSUP;
            break;
```

Parent topic: Networking Protocol Restrictions and Optimizations on page 27

10.2. Driver Checksum Offloading and Receiving

Your driver's receive function is responsible for certain tasks related to checksum offloading on the receiving side.

About This Task

The driver receive function must do the following:

Procedure

1. Determine if the network stack has requested that the device-calculated checksum be passed to the stack.

   If **IFCAP_RXCSUM** is set in the **cap_enabled** field in driver's copy of the **END_CAPABILITIES** structure, the stack has requested that the checksum be passed.

   If receive checksumming is enabled, the driver reads the device's checksum status report for the packet.

2. If the device calculated the IP header checksum, the driver sets **CSUM_IP_CHECKED** in the packet **mBlk->mBlkPktHdr.csum_flags** to indicate that the IP header checksum was calculated.

   Note: The Wind River Network Stack does not make use of the **CSUM_IP_CHECKED** or **CSUM_IP_VALID** bits; it always calculates the IPv4 header checksum in software.

3. If the IP header checksum is valid, the driver sets **CSUM_IP_VALID** in the packet **mBlk->mBlkPktHdr.csum_flags** to indicate that the IP header is valid.
4. The driver tests if the device calculated the TCP or UDP checksum and if the checksum is valid, which indicates that the packet is uncorrupted.
   1. If the device calculated the TCP or UDP checksum and determined that the packet is valid, the driver sets **CSUM_DATA_VALID** in the packet **mBlk->mBlkPktHdr.csum_flags**.
   2. If the device also computed the pseudo header, the driver sets **CSUM_PSEUDO_HDR** in the packet **mBlk->mBlkPktHdr.csum_flags**.
   3. If the driver determines that the packet and checksum are valid, it writes the checksum into the M_BLK at **pMblk->m_pkthdr.csum_data**.

   Your driver does not need to read the calculated checksum from a device register. A valid checksum value is 0xffff, the driver can write this value into the M_BLK. However, the value 0xffff can only be used if the device calculates the checksum over the entire TCP segment or UDP datagram, including the pseudo-header (**CSUM_PSEUDO_HDR** must be set). If the checksum calculated by the device does not include the pseudo-header, the value put into **pMblk->m_pkthdr.csum_data** must be the actual un-complemented ones complement sum over the segment or datagram not including the pseudo-header.

Parent topic: Networking Protocol Restrictions and Optimizations on page 27

10.3. Corrupt Packet Handling

You can use your device hardware to indicate that a transport checksum is good. However, your driver must verify any device-reported invalid checksums.

Barring known errata, you can trust a device to indicate when a checksum is good, but not to indicate when a checksum is bad. Wind River recommends that if your device reports a packet to have an invalid transport checksum, your driver should treat the packet in the same manner it would treat the packet when the device fails to report any checksum status (that is, do not set **CSUM_DATA_VALID**). In this case, the stack calculates the packet checksum in software. This approach allows you to take advantage of some checksum offload benefit even for devices that in certain (rare) instances, incorrectly report a packet as having a bad checksum. Test a driver that supports checksum offload on reception by sending it packets with a variety of bad checksums as well as good ones, to validate that the driver can distinguish the good from the bad correctly. If a device reports some bad checksums as good, do not enable checksum offload in the system.

A driver supporting checksum offload might have code like the following in its receive handler:

```
/* Do RX checksum offload, if enabled. */

if (pDrvCtrl->hwCaps.cap_enabled & IFCAP_RXCSUM)
    {
    /* Read the device checksum status register */
    RFD_BYTE_RD (pRbdTag->pRFD, RFD_CSUMSTS_OFFSET, csumStatus);

    /* Determine if IP checksum calculated */

    if (csumStatus & RFD_CS_IP_CHECKSUM_BIT_VALID)
        {
         /* Set mBlk check sum flags to indicate checksum calculated */
         pRbdTag->pMblk->m_pkthdr.csum_flags |= CSUM_IP_CHECKED;
        }

    /* Determine if IP checksum valid

    if (csumStatus & RFD_CS_IP_CHECKSUM_VALID)
        {
         /* Set mBlk check sum flags to indicate a valid IP header */
         pRbdTag->pMblk->m_pkthdr.csum_flags |= CSUM_IP_VALID;
        }

    if (csumStatus & RFD_CS_TCPUDP_CHECKSUM_BIT_VALID &&
        csumStatus & RFD_CS_TCPUDP_CHECKSUM_VALID)
        {
         pRbdTag->pMblk->m_pkthdr.csum_flags |=
             CSUM_DATA_VALID|CSUM_PSEUDO_HDR;
         pRbdTag->pMblk->m_pkthdr.csum_data = 0xFFFF;
        }
    }
```

Parent topic: Networking Protocol Restrictions and Optimizations on page 27

11. Network Interface Driver Receive Handling

To improve performance, Wind River recommends that you follow certain guidelines when implementing your driver's receive ISR and the task-level receive handler function queued from the ISR.

VxWorks network drivers defer much of the work related to servicing interrupt conditions to code executing at task level, by calling jobQueuePost( ).

Because interrupts are relatively costly in terms of overall system performance, one goal of a network interface driver is to minimize the number of interrupts that occur, especially when under heavy load. To accomplish this goal, consider the following guidelines:

- When the device receive ISR runs, it must disable further receive interrupts from the device, and queue the task-level receive handler for execution using jobQueuePost( ).
- The network task runs the task-level receive handler function, processing received packets until there are no more to handle. Then, the receive handler function re-enables receive interrupts from the device, and returns.

> Note: If the device uses a single interrupt to report not just received packets (or overruns) but also non-receive events such as transmit completions and link state changes, then the same strategy can be used, employing a single task-level handler function that handles receive packets, transmit completions, and link state changes as needed.

Under a heavy load of received packets, the task-level receive handler function services many packets before re-enabling interrupts from the device.

This simple approach needs certain refinements to deal with the following two primary complications:

- If the interrupt line used by the device is shared by other devices, the device's ISR can in fact run even when the device has interrupts disabled.
- The network task that runs the task-level receive handler function, most often **tNet0**, is shared by the network stack itself and potentially many network devices. The network task runs queued jobs one by one, finishing the current job before starting the next one. For fairness to other network devices, and to other types of network jobs such as transmit cleanup handling and protocol-level timeout processing, a driver's receive handler must limit the amount of work it does in any one network job execution queued with jobQueuePost( ).

Consider the problem of shared interrupt lines first. Assume that you have two devices, A (the network device) and B (some other device, possibly another network device), that share an interrupt line. The result of the sharing is that, whether A or B generated a particular interrupt, the ISRs of device A and device B both run as a result, even if one of the devices has interrupts disabled. Therefore, simply disabling device interrupts cannot prevent a device ISR from running if the interrupt line is shared.

The conventional strategy for ISRs in environments where this is possible is that each ISR first reads a status register of its device to see whether there is an event pending on the device that might have caused the interrupt. If not, the ISR quickly exits; it knows that its device did not cause the interrupt. The other device's ISR presumably finds that its device did cause the ISR and handles it appropriately. However, there are some possible problems with this approach. For example:

- Many devices show an event (for example, a received packet) even if interrupts are currently disabled for that device, and the task-level handler job is already posted.
- For some devices, reading the event status register can have the side effect of acknowledging any events found. If the task-level handler is already posted, you can lose the acknowledged events, causing them to (possibly) not be handled until a subsequent event occurs. (For example, a later packet arrives.)
- A driver typically uses a single **QJOB** member embedded in its control structure for the device instance to post the receive handling job. If the **QJOB** is currently enqueued waiting to run, posting it a second time has no effect. However, you could lose information about the particular event that elicited the post. Alternatively, if the handler has already started to run (and has not reposted itself), posting the **QJOB** using jobQueuePost( ) causes it to be queued again for execution. Before the current instance of the handler returns, it can re-enable device interrupts if it finds no more work to do. This can result in the new handler instance running with device interrupts enabled. While this is not necessarily catastrophic, this condition can persist for

a long time interval, depending upon how the ISR is written and how much load there is, leading to increased CPU utilization during that interval.

You want to prevent an ISR that runs due to interrupt line sharing from reposting the receive handler job when it is already posted or executing. This avoids the 'handler running with interrupts enabled' issue mentioned previously, and unnecessary work. To achieve this, There must be some synchronization between the ISR and the task level handler (because you cannot rely upon disabling device interrupts alone). If not carefully done, such synchronization is susceptible to various race conditions, from lost events (with the typical result that some packets occasionally languish in the receive ring until a subsequent packet arrives), to permanent suspension of receive processing, or even work queue panics.

Wind River recommends two methods that work well for most devices when synchronizing between the device ISR and the task-level handler. Both use a flag that indicates whether the task-level handler has already been posted; if the ISR runs and finds this flag already set, it does not re-post the handler job. If it finds the flag clear, it disables interrupts and posts the task-level handler. The handler job is responsible for clearing the flag and re-enabling device interrupts just before it returns. The two methods differ in what constitutes the "flag", and in what code acknowledges device interrupt events.

The first method is the simpler of the two; it makes use of the device interrupt mask register (**MY_IMR** in the code examples that follow) as the flag to detect whether interrupts are already masked and the task-level handler job already posted. This method also avoids acknowledging device interrupt events in the ISR; only the task-level handler acknowledges interrupt events. For this method, the device ISR and task-level handler look similar to the following example code.

The second method is more complex. This method uses an atomic variable to record the interrupt events that the interrupt service routine acknowledges, as well as to flag whether or not the task-level handler has been posted. It can potentially be preferable to method 1 in cases where the interrupt status register cannot be read without acknowledging events. It is also more easily adaptable to certain hardware for which transmit and receive interrupts cannot be disabled for an individual device.

This example assumes that you have a device that uses a single ISR for all event types of interest (for example, receive packet arrival or overrun, transmit completion, link state change, and so forth), and posts a single handler job to deal with any of the events. However, you can modify the example to suit a device that uses separate receive, transmit, and link state ISRs.

```
/*****************************************************************************
 *
 * myEndInt - Interrupt Service Routine to handle device interrupts
 */

LOCAL void myEndInt
    (
    MY_DRV_CTRL * pDrvCtrl
    )
    {
    VXB_DEV_ID pDev;
    UINT32 status;

    pDev = pDrvCtrl->myDev;

    /*
     * Read interrupt status register to check for pending device events.
     * Here we assume that this does not acknowledge any such events.
     */

    status = CSR_READ_4(pDev, MY_ISR);

    /*
     * Make sure there's really an interrupt event pending for us.
     * We assume that this device may share an interrupt line
     * with another device. If so, this function might be invoked
     * as a result of the other device asserting the interrupt, in
     * which case we might not have any work to do. MY_INTRS is
```

```
     * a mask of the interrupt events that we are interested in.
     */

    if ((status & MY_INTRS) == 0)
        return;

    /*
     * Check the device's interrupt mask register (MY_IMR).
     * If device interrupts are already disabled (masked),
     * we are either stopped, or have already posted a job to handle
     * previous events, and it hasn't finished running yet. Don't post
     * another job in that case, just return.
     */

    if (CSR_READ_4(pDev, MY_IMR) == 0)
        return;

    /*
     * Otherwise, disable device interrupts here by clearing
     * the mask register
     */

    CSR_WRITE_4(pDev, MY_IMR, 0);

    /*
     * This atomic flag is not used here to synchronize between the
     * ISR and the task-level handler. It merely lets the driver stop
     * routine know that the handler is pending (or running), which
     * helps with clean shutdown.
     */

    (void)vxAtomic32Set (&pDrvCtrl->handlerPending, TRUE );

    /* post the handler job */

    (void)jobQueuePost (pDrvCtrl->myJobQueue, &pDrvCtrl->myIntJob);

    return;
    }

/*******************************************************************************
 *
 * myEndIntHandle - task level handler job for RX/TX/Link interrupts
 *
 */

LOCAL void myEndIntHandle
    (
    void * pArg
    )
    {
    QJOB *pJob;
    MY_DRV_CTRL *pDrvCtrl;
    VXB_DEV_ID pDev;
    UINT32 status;

    /*
```

```
     * Convert the QJOB argument to a pointer to the driver control
     * structure for this device instance.
     */

    pJob = pArg;
    pDrvCtrl = member_to_object (pJob, MY_DRV_CTRL, myIntJob);
    pDev = pDrvCtrl->myDev;

    /*
     * Read and acknowledge interrupts here. For this device, we
     * acknowledge the interrupts by writing the value read back to
     * the interrupt status register.
     */

    status = CSR_READ_4(pDev, MY_ISR);
    CSR_WRITE_4(pDev, MY_AUXISR, status);

    /*
     * Check if we need to do RX handling work. We do if there
     * were RX events pending, or if on the previous execution
     * we handled the maximum number of packets per job.
     */

    if ((status & MY_RXINTRS) || pDrvCtrl->moreRx)
        {
        pDrvCtrl->moreRx = FALSE;
        /*
         * myEndRxHandle() handles a bounded number of received packets,
         * and sets pDrvCtrl->moreRx if it has to stop because it reaches
         * the maximum number per job before running out.
         */
        myEndRxHandle (pDrvCtrl);
        }

    /* Do transmit cleanup work if needed. */

    if (status & MY_TXINTRS)
        {
        myEndTxCleanup (pDrvCtrl); /* free all completed TX resources */
        }

    /* Handle link state changes if necessary. */

    if (status & MY_LINKINTRS)
        myLinkUpdate (pDev);

    /* Check for additional interrupt events */

    if (pDrvCtrl->moreRx || CSR_READ_4(pDev, MY_ISR) & MY_INTRS)
        {
        jobQueuePost (pDrvCtrl->myJobQueue, &pDrvCtrl->myIntJob);
        return;
        }

    vxAtomic32Set (&pDrvCtrl->myIntPending, FALSE);

    /* re-enable interrupts here */
```

```
    CSR_WRITE_4(pDev, MY_IMR, pDrvCtrl->myIntrs);

    return;
    }
```

In this example, all of the event bits that the driver is interested in (**MY_INTRS**) fit in a 32-bit atomic variable, and there is at least one available bit left over (**MY_HANDLER_PENDING**) that can be used as a flag to indicate that the task-level handler has been posted.

```
/*******************************************************************************
 *
 * myEndInt - Interrupt Service Routine to handle device interrupts
 */

LOCAL void myEndInt
    (
    MY_DRV_CTRL * pDrvCtrl
    )
    {
    VXB_DEV_ID pDev;
    UINT32 status;

    pDev = pDrvCtrl->myDev;

    /* Read the interrupt status register */

    status = CSR_READ_4(pDev, MY_ISR);

    /*
     * Make sure there's really an interrupt event pending for us.
     * We assume that this device may share an interrupt line
     * with another device. If so, this function might be invoked
     * as a result of the other device asserting the interrupt, in
     * which case we might not have any work to do. MY_INTRS is
     * a mask of the interrupt events that we are interested in.
     */

    status &= MY_INTRS;
    if (status == 0)
        return;

    /*
     * Acknowledge interrupts by writing back the same bits.
     * For a few devices, the acknowledgement happens automatically
     * as a side-effect of the register read above, and this
     * write is not needed.
     */

    CSR_WRITE_4(pDev, MY_ISR, status);

    /*
     * Save the event bits that we acknowledged for use by
     * the task-level handler, and find out if we need to
     * post the handler job.
     */

    status = vxAtomic32Or (&pDrvCtrl->eventsPending,
```

```
                        status | MY_HANDLER_PENDING);

    /*
     * vxAtomic32Or() returns the value of eventsPending before the
     * atomic OR operation. If the handler was already pending,
     * do nothing else. Otherwise, disable device interrupts and
     * post the job to execute the task level handler.
     */

    if (status & MY_HANDLER_PENDING)
        return;

    /* Disable device interrupts for the device. */

    CSR_WRITE_4(pDev, MY_IMR, 0);

    /* post handler job */

    jobQueuePost (pDrvCtrl->myJobQueue, &pDrvCtrl->myIntJob);

    return;
    }

/*******************************************************************************
 *
 * myEndIntHandle - task level handler job for RX/TX/Link interrupts
 *
 */

LOCAL void myEndIntHandle
    (
    void * pArg
    )
    {
    QJOB *pJob;
    MY_DRV_CTRL *pDrvCtrl;
    VXB_DEV_ID pDev;
    UINT32 status;

    /*
     * Convert the QJOB argument to a pointer to the driver control
     * structure for this device instance.
     */

    pJob = pArg;
    pDrvCtrl = member_to_object (pJob, MY_DRV_CTRL, myIntJob);
    pDev = pDrvCtrl->myDev;

    /*
     * Read the current events pending saved by the ISR, and atomically
     * clear all bits in eventsPending except for MY_HANDLER_PENDING.
     */

    status = vxAtomic32And (&pDrvCtrl->eventsPending, MY_HANDLER_PENDING);

    /*
     * Check if we need to do RX handling work. We do if there
```

```
     * were RX events pending, or if on the previous execution
     * we handled the maximum number of packets per job.
     */

    if ((status & MY_RXINTRS) || pDrvCtrl->moreRx)
        {
        pDrvCtrl->moreRx = FALSE;
        /*
         * myEndRxHandle() handles a bounded number of received packets,
         * and sets pDrvCtrl->moreRx if it has to stop because it reaches
         * the maximum number.
         */
        myEndRxHandle (pDrvCtrl);
        }

    /* Do transmit clean-up work if needed. */

    if (status & MY_TXINTRS)
        {
        myEndTxCleanup (pDrvCtrl); /* clean up all completed TX */
        }

    /* Handle link state changes if necessary. */

    if (status & MY_LINKINTRS)
        myLinkUpdate (pDev);

    /*
     * Check for additional relevant interrupt events in status
     * register MY_ISR.
     */

    status = CSR_READ_4(pDev, MY_ISR) & MY_INTRS;

    /*
     * Acknowledge any events by writing back the same bits.
     * For a few devices, the acknowledgement happens automatically
     * as a side-effect of the read above, and this write is not
     * needed.
     */

    CSR_WRITE_4(pDev, MY_ISR, status);

    /*
     * Check also for additional events saved by the myEndInt(),
     * running as a result of sharing the interrupt line, and
     * save any we may have read above.
     */

    status |= vxAtomic32Or (&pDrvCtrl->eventsPending, status);

    /*
     * Repost this job if there were more events to handle, or if the RX
     * handler routine reached the maximum number of packets per job.
     */

    if (pDrvCtrl->moreRx || (status & MY_INTRS) != 0)
```

```
        goto repost;

    /* Re-enable device interrupts */

    CSR_WRITE_4(pDev, MY_IMR, pDrvCtrl->myIntrs);

    /*
     * If myEndInt has saved no more events for us, atomically
     * clear MY_HANDLER_PENDING, which would be the only bit on
     * in eventsPending. Otherwise we lost the race, and
     * we must remask interrupts and repost ourself.
     * We use an atomic compare-and-swap operation for this.
     * vxAtomic32Cas() returns TRUE if the write actually occurs, FALSE
     * otherwise.
     */

    if (!vxAtomic32Cas (&pDrvCtrl->eventsPending, MY_HANDLER_PENDING, 0 ))
        {
        CSR_WRITE_4(pDev, MY_IMR, 0);
repost:
        jobQueuePost (pDrvCtrl->myJobQueue, &pDrvCtrl->myIntJob);
        }

    return;
    }
```

Either of the two methods presented above can be applied to any number of devices. Device hardware is diverse and these methods may not work as presented in all cases. However, you can use them as starting points for further development. For example, on some hardware platforms, shared interrupt lines are not possible, in which case drivers restricted to those platforms can use somewhat simpler code in the ISR(s) and task-level handler(s). Another variation is that in some cases it is necessary to use a spinlock to protect non-atomic accesses to device registers from the ISR and the task-level handler. In this case, given that the spinlock is already used, you can use ordinary variables protected by the spinlock, rather than atomic variables, to flag that the handler is running, to save acknowledged events, and so on.

Both of the examples rely on a separate receive handler function that limits the number of packets it delivers to the MUX in one execution (and indicates to the caller by some means if it reaches that limit). (A typical limit on the number of received packets delivered per job would be 16 or 32 packets.) This helps to ensure that a single network job cannot monopolize the network task (usually **tNet0**) for too long, preventing other work that needs to be done. For example, when forwarding packets between two interfaces, doing transmit cleanup work on the egress interface with sufficient frequency is just as necessary to good performance as delivering received packets from the ingress interface. If the receive handler job for the ingress interface executes for too long, the job that does transmit cleanup for the egress interface may not get to execute, so that the egress transmit ring fills up and packets start being dropped, damaging performance.

One technique that seems to help performance somewhat is to have the receive handler collect available packets from its receive (RX) ring (up to its limited quota) before delivering them to the stack, rather than delivering each packet as it is removed from the RX ring. This means the driver receive code and data structures are more likely to stay cache-hot which opens up greater opportunities for intelligent pre-fetching of packet data.

In principle, the amount of transmit cleanup work done within a single network job execution ought to be bounded like the amount of packet receive work . However, transmit cleanup work is usually much faster than the packet processing done on the receive side. Also, the transmit ring is not added to while transmit cleanup is occurring, and is generally small enough that even if the full transmit ring is cleaned in a single job, it is not an excessive amount of work.

12. MAC Receive Stall Handling

Your driver must be prepared to handle a receive stall condition. How you handle this in your driver, depends on the behavior of your device hardware.

When a device receives packets, it copies each packet's data into a buffer (cluster) provided by the driver. The device can continue to deliver received packets in this way even as the device driver code executes to process the packets and replenish the buffers. However, the driver can encounter a situation where it receives enough packets to fill the available clusters before the it has a chance to process the received packets and make additional clusters available. When this happens, the device stops copying into the receive clusters. This condition is known as a *receive stall*.

When this occurs, devices typically behave in one of two ways:

- Some devices require that the next descriptor in the sequence be cleared, and no additional driver intervention is required. That is, the descriptor's status must be set to free or available. In this case, the device automatically detects that the stall is cleared and resumes operation without any other action on the part of the driver.
- Other devices place their receiver into a halted state by setting a bit in a control register. This type of device often requires the driver to clear the control register bit in addition to freeing the next descriptor, before operation resumes.

Be sure that your driver's receive handler function handles the case of a receive stall in a way that is appropriate for the device.

13. MAC Driver Debugging Methods

In addition to standard driver debugging strategies, there are several strategies you can employ when debugging your network interface (MAC) driver.

When working with a VxWorks network interface (MAC) driver, you must have some way to boot the VxWorks image without using the driver you are attempting to debug. You can do this by using some other network device on the target hardware, or you can use an image programmed into ROM by some form of hardware debugger.

Debugging Network Interface Drivers Using VxBus Show Functions on page 42
Deferring Driver Registration on page 43 When performing network interface device driver debugging, you can defer the device driver registration with VxBus.
Stress Testing on page 44 To create reliably network drivers, Wind River strongly recommends the use of hardware debug tools and third-party software suites for stress testing.
Interrupt Validation Testing on page 44 You must be sure interrupts are being generated correctly. This can be tested using the logMsg( ) function.
Additional Tests for MAC Drivers on page 44 Once your driver provides basic functionality, there are a number of additional tests that can be run. Many of these tests can be used without special hardware or software platforms.

13.1. Debugging Network Interface Drivers Using VxBus Show Functions

Network driver debugging makes more use of the VxBus show functions than any other driver class. These functions are used in the usual way to find whether the driver matches the device, to find whether the device exists, and so on. However, because PHY device configuration is sometimes performed as part of the network device initialization, PHY debugging is also relevant to network device initialization and VxBus show functions can be used to help some kinds of PHY debugging. For example, you can use vxbDevShow( ) to find whether the appropriate PHY device is connected.

The following show functions are useful when debugging:

vxbDevShow( )

Shows all discovered devices (based on scan a of the flattened device tree (FDT), PCI, or MII bases) in a hierarchical order.

vxbDrvShow( )

Shows all of the device drivers that are registered with the kernel (including drivers that are not assigned to any device).

You can also add your own device-specific show functions into your device driver code and then invoke them through the shell.

Parent topic: MAC Driver Debugging Methods on page 42

13.2. Deferring Driver Registration

When performing network interface device driver debugging, you can defer the device driver registration with VxBus.

As with all VxBus drivers, it is helpful to defer driver registration when debugging network drivers. However, in order for the driver to work, you must connect the instance to the MUX and the network stack. In a normal system, these actions are done automatically, once, during system startup, and never done again. Therefore, during testing, you must perform these actions manually.

Because the attach method both initializes the device and connects it to the MUX, you must prevent VxBus from registering the driver during the single VxBus initialization phase. Once the system is running, you must manually register the device driver. This registration automatically triggers the probe and attach methods of the device driver which in turn initialize the device and connect it to the MUX.

Note: During your VxWorks image project (VIP) configuration, you must configure your kernel to support the kernel shell and use a standalone symbol table. This allows you to invoke functions from the shell.

To prevent your device driver from loading at system startup, remove or comment out the following code in your device driver:

```
VXB_DRV_DEF(vxbMyEnetDrv)
```

Next, add a driver register function. For example:

```
void addMyEnetDrv()
{
  vxbDrvAdd(&vxbMyEnetDrv);
}
```

Rebuild the VxWorks layer that contains your driver or rebuild the entire VSB, and rebuild your VIP

After booting, call the driver register function (addMyEnetDrv( )) from the shell. This registers the driver, initializes the device, and attaches it to the MUX.

Attaching to the IPv4 Stack

To attach to the IPv4 stack, use a sequence of calls to ipcom_drv_eth_init( ) and ifconfig( ) to configure the device and attach to the stack. The ipcom_drv_eth_init( ) function uses three arguments: the driver name, the unit number, and a third argument that is always zero. The ifconfig( ) function is similar to ifconfig( ) on UNIX and similar operating systems. However, you must include the argument list in quotes. (For a more detailed discussion of the arguments to ifconfig( ), see the related reference entry.)

Note: The ifconfig command is also available from the command shell interpreter, where it is called without quotes around the argument list.

For example, if connecting **GEI0** to IPv4 at address 10.0.0.1, use the following commands from the VxWorks development shell:

```
-> ipcom_drv_eth_init("gei", 0, 0)
-> ifconfig("gei0 10.0.0.1 netmask 255.255.255.0 up ")
```

Alternatively, you can use the ifconfig command from the kernel command shell. This method avoids having to use the string argument.

Note: You must include the **INCLUDE_IPIFCONFIG_CMD** component in your VxWorks image project (VIP) in order to use the kernel command shell version of ifconfig.

Parent topic: MAC Driver Debugging Methods on page 42

13.3. Stress Testing

To create reliably network drivers, Wind River strongly recommends the use of hardware debug tools and third-party software suites for stress testing.

Using a debug tool, such as SmartBits or IXIA, generally allows you to create a better high stress environment for testing and generally leads to a more reliable and robust driver. In many cases, it is not possible to create an adequate test environment without the use of hardware debug tools.

Parent topic: MAC Driver Debugging Methods on page 42

13.4. Interrupt Validation Testing

You must be sure interrupts are being generated correctly. This can be tested using the logMsg( ) function.

During early debugging, instrument the driver's ISR by adding an output message using logMsg( ). This lets you know if the device generates interrupts correctly and if the ISR is connected correctly. It also lets you know what types of interrupts are occurring and how the interrupts are being processed.

Parent topic: MAC Driver Debugging Methods on page 42

13.5. Additional Tests for MAC Drivers

Once your driver provides basic functionality, there are a number of additional tests that can be run. Many of these tests can be used without special hardware or software platforms.

Ping-of-Death

Ping-of-death is an attack on drivers based on the value of an unsigned 16-bit field in the ping packet. When the ping packet size is larger than 32 KB (32768 bytes), some drivers and network stacks cannot handle the packet. To generate a ping-of-death, specify a 64 KB ping packet using any ping client.

Driver Start and Stop

Test starting and stopping your driver. To stop the driver, use ifconfig( ) with the down argument and muxDevStop( ). The argument to muxDevStop( ) is the cookie returned by the muxDevLoad( ) call when you initialize your driver. You can also obtain the MUX device cookie for your driver by calling the endFindByName( ) function. For example:

```
-> geiCookie = endFindByName ("gei", 0)
-> ifconfig ("gei0 down")
-> muxDevStop(geiCookie)
```

To restart the driver, call muxDevStart( ) and call ifconfig( ) with the up argument. For example:

```
-> muxDevStart(geiCookie)
-> ifconfig("gei0 up")
```

Polled Mode

You must test your driver's polled mode operation. Typically, the only use of polled mode is for core dumps. You must perform some basic testing. To do this, write a simple application that makes the ioctl( ) call to put the instance into polled mode operation. The application must read from the interface using polled mode, modify the packet data, and send the modified packet. This can be tested using ping from your development host. When polled mode is used to save core dump data, the network is put under heavy load. For this reason, run the basic testing with heavy traffic. For example, run ping with large packet sizes from multiple hosts, to ensure that many large packets can be received and transmitted. Additional testing can be done by performing an actual core dump save. For more information on core dump, see the *VxWorks 7 Debugging Guide: About This Guide*.

Receive Error Path

Be sure to test the receive error path for your driver. This testing is often overlooked when hardware debug tools are not available, because it is difficult to generate receive error conditions without those tools. You cannot fully validate the receive error path without the assistance of hardware tools.

Multicast Filter Test

If your driver supports multicast, test to see whether the multicast filter works correctly. To do this, you must write a simple test application that runs on VxWorks and which configures the interface with one or more multicast addresses and sends and receives multicast traffic. You may need host software or multiple VxWorks targets in order to perform this type of test.

Parent topic: MAC Driver Debugging Methods on page 42

3. PHY DRIVERS

1. PHY Driver Overview

All 10/100/1000 Ethernet interfaces incorporate a physical layer of some type, commonly known as a PHY. The VxWorks MII bus and PHY driver mechanism provides both a simple way for Ethernet drivers to handle link management, and for different PHY chips to be handled with discrete, reusable drivers.

The PHY component can be integrated directly with the MAC, or it may be a separate device connected to the MAC using one of several MAC/PHY media connection types (such as MII, GMII, RGMII, TBI, and so forth). Software interaction with the PHY is necessary in order to properly implement link auto-configuration and link state change notification within VxWorks.

The IEEE 802.3 specification defines two kinds of management interfaces for PHY devices, which are typically referred to as "clause 22" and "clause 45" (so named because of the clauses in the specification that describe them). Both interfaces use a two-wire communication scheme between the host and PHY (using one clock pin and one data pin).

Most 10 Mbps, 100 Mbps and 1000 Mbps network devices use the clause 22 management protocol, which allows for 32 physical devices per 2-wire management bus, each with 32 16-bit registers. The first 16 are defined by the 802.3 specification itself and are common to all devices that are 802.3-compliant. The remaining 16 registers are vendor-defined, and vary from one device to another.

For network interfaces that support speeds of 10 Gbps or greater, the clause 45 protocol is used. It also supports 32 devices per 2-wide management bus, but each device can have 32 sub-devices (known as MDIO Managable Devices, or MMDs), and each sub-device can have up to 65536 registers. The first 30 MMDs are defined by the 802.3 specification and the remaining two are vendor-defined.

While it is possible to use only the standardized registers to control most devices, there are many cases in which use of the vendor-specific API is required. In these cases, it is necessary to implement device-specific PHY software.

Because the clause 22 and clause 45 read and write transaction protocols differ, a single MII management bus will not contain both clause 22 and clause 45 devices. A single 2-wire bus will have either all clause 22 devices or clause 45 devices, but not both. Clause 22 and clause 45 PHY and MDIO devices typically require separate drivers.

The MII bus and PHY layer includes the **vxbMiiLib.o** module (which is configured into the system using the INCLUDE_MII_BUS component) and various PHY drivers. A given image configuration need only include those PHY drivers that are required to handle the PHY hardware actually present in the system. In many cases, only the **genericPhy** driver is necessary.

The link management functions of the MII bus are carried out in the context of the **miiBusMonitor** task. This task periodically checks the state of every interface configured into the system and issues a callback to the corresponding VxBus MAC instance whenever the link state changes.

Currently, PHY interrupts are not supported. To understand why, consider that acknowledging and cancelling a PHY interrupt requires reading or writing to a PHY register, and that to correctly follow VxWorks driver guidelines, this operation must be done in an ISR. However, PHY register accesses are typically done indirectly through an MDIO port and are not atomic. This means that they must use mutual exclusion protection to prevent overlapping accesses. The problem with this is that the only mutual exclusion mechanism that works in both task context and interrupt context is intLock( ) and intCpuLock( ) (or a spinlock, if you are using VxWorks SMP), but in some cases a PHY register access is slow, and keeping interrupts blocked for the entire period can negatively impact system behavior. (This is especially true where MDIO accesses are performed using serial bitbang I/O in software.)

Consequently, PHY register accesses are never done in interrupt context, and polling is used to monitor link state changes instead. The current polling period is two seconds, and **miiBusMonitor** task runs at priority 252, in order to reduce load on the system as much as possible.

MAC and MII Bus Relationship on page 47 Each Ethernet MAC driver typically has one MII bus. This bus in turn has one or more child PHY devices attached to it. Your driver design must be carefully tailored to your specific hardware system.

Generic PHY Driver Support on page 47 VxBus provides a generic PHY driver (**genericPhy**) is included with the MII bus subsystem and is designed to support most 10/100/1000 Mb/s copper PHYs.

1.1. MAC and MII Bus Relationship

Each Ethernet MAC driver typically has one MII bus. This bus in turn has one or more child PHY devices attached to it. Your driver design must be carefully tailored to your specific hardware system.

The simplest and most common case is one Ethernet device instance, with one child MII bus, with one child **genericPhy** instance.

Originally, the reason for supporting multiple PHYs on a single MII bus was to allow for the design of network controllers with more than one media type. For example, a dual media copper and fiber Ethernet adapter could be built using one Ethernet MAC with two different PHYs (one copper and one fiber). In this example, the driver software sets the interface for copper mode by using the MII management interface to isolate or power down the fiber PHY while activating the copper one. Switching to fiber mode is done using the opposite procedure (isolating the copper PHY and then re-enabling the fiber one). In this configuration, only one PHY is active at a time, and the idle one is isolated from the MAC data pins.

This configuration is not commonly used (several vendors now support both copper and fiber media in a single PHY chip instead, using vendor-specific programming to switch modes). However, what is more common is the use of a single MDIO port for controlling multiple PHYs connected to different MACs. For example, the Freescale MPC8560 has two built-in tsec gigabit Ethernet MACs. However, only one of them has a functional MDIO port. This means that software can only access the management registers on the two PHY chips by using the MDIO registers on only one of the tsecs (typically **TSEC0**).

This configuration presents a problem, because it can result in the MII bus for one tsec device having two child PHY instances, while the MII bus on the other tsec has none. Your Ethernet MAC driver software must be carefully written in order to deal with this condition.

Parent topic: PHY Driver Overview on page 46

Related information

Generic PHY Driver Support on page 47

1.2. Generic PHY Driver Support

VxBus provides a generic PHY driver (**genericPhy**) is included with the MII bus subsystem and is designed to support most 10/100/1000 Mb/s copper PHYs.

The generic PHY driver uses only those registers defined in the MII specification for controlling the underlying PHY device, and works with the majority of PHY chips without modification. The **genericPhy** driver probe function always succeeds, and acts as a "catch-all" for any PHYs discovered on an MII bus that are not specifically claimed by another driver registered with VxBus.

The generic PHY driver always assumes that the PHY device supports at least 10 Mb/s and 100 Mb/s modes in full and half duplex. It also tests for the extended capabilities bit in the status register and, if this bit is set, it enables support for autonegotiation as well.

Include the **genericPhy** driver in your VxWorks image project (VIP) using the INCLUDE_GENERICPHY component.

Parent topic: PHY Driver Overview on page 46

Related information

MAC and MII Bus Relationship on page 47

2. Device Driver Methods for PHY Drivers

The MII bus layer has two sets of methods: upper edge methods, which must be provided by Ethernet MAC drivers, and lower edge methods, which must be provided by child PHY devices that reside on an MII bus.

Note: These methods typically perform operations that are not atomic. In particular, performing MDIO reads and writes usually requires multiple accesses to MAC registers. Consequently, it is important that these functions internally provide some form of mutual exclusion in order to prevent overlapping accesses. Most VxBus Ethernet drivers do this using a mutex semaphore.

Upper Edge Methods

All upper edge MII bus methods are typically exported by the MAC driver that instantiates the MII bus. In most cases, access to the PHY management registers is provided through an MDIO port that is part of the Ethernet MAC itself. This can either be a low level bitbang interface to the MDIO pins, or it can be a set of "shortcut" registers that permit read and write accesses to the PHY, while the MAC hardware implements the bitbang MDIO protocol internally. The **miiBus** code must be able to read and write to these management registers, therefore the MAC driver must export read and write methods to **miiBus**.

Many MACs must be explicitly programmed to match the link speed (10, 100, or 1000 Mb/s) and the duplex state (full or half) of the PHY in order to function correctly. A MAC driver for such a device must be notified when the link state changes, so that it can synchronize its state with that of the PHY. To do this, the MAC driver must export a link update method, {miiLinkUpdate}( ), so that the **miiBusMonitor** task (or a network task, in the case of a link state change interrupt event) can notify the MAC driver when the link state changes.

When a MAC driver publishes the {miiLinkUpdate}( ) method, it usually publishes the {miiRead}( ) and {miiWrite}( ) methods as well. These methods are always called from task context. For more information on these methods, see VxBus Device Driver Methods for MAC Drivers on page 5.

Lower Edge Methods

The lower edge MII bus methods are exported by PHY drivers only. Currently, there are only two methods available: {miiModeGet}( ) and {miiModeSet}( ). These methods are used to get and set the PHY media mode and are used internally by the miiBusModeGet( ) and miiBusModeSet( ) functions provided by **miiBus**. Each PHY driver must export these methods in order for the **miiBus** code to properly use the driver to manage the link.

{miiModeGet}( )

The {miiModeGet}( ) method is declared as follows:

```
LOCAL STATUS miiModeGet
    (
    VXB_DEV_ID pDev,
    UINT32 * mode,
    UINT32 * status
    )
;
```

The {miiModeGet}( ) method is used to return the current link state information. The pDev is a pointer to the PHY instance context. The mode and status arguments point to storage where the {miiModeGet}( ) method returns the current link settings, and the link status. The mode value is specified in terms of the macros defined in the **endMedia.h** header file. The status field sets the **IFM_AVALID** bit if it contains valid data, and the **IFM_ACTIVE** bit is set if the link is up.

Note: The {miiModeSet}( ) method must be called at least once to initialize the PHY before the {miiModeGet}( ) method is used to check the link state.

{miiModeSet}( )

The {miiModeSet}( ) method is declared as follows:

```
LOCAL STATUS miiModeSet
    (
    VXB_DEV_ID pDev,
    UINT32 mode
    )
;
```

The {miiModeSet}( ) method is used to set the PHY link to a particular mode. The pDev argument is a pointer to the PHY instance context. The mode argument is an encoded value using the definitions from the **endMedia.h** header file. The following example illustrates how to decode the mode value to obtain the speed and duplex values:

```
switch(IFM_SUBTYPE(mode)) {
case IFM_AUTO:
    /* Autonegotiation */
    break;
case IFM_1000_SX:
    /* 1000baseSX, fiber */
    break;
case IFM_1000_T:
    /* 1000baseT, copper */
    break;
case IFM_100_TX:
    /* 100baseTX, copper */
    break;
case IFM_10_T:
    /* 10baseT, copper */
    break;
default:
    return (ERROR);
    break;
}

if ((mode & IFM_GMASK) == IFM_FDX)
    /* full duplex mode */
else
    /* half duplex mode */
```

Prior to setting the link state, the func{miiModeSet}( ) function should also reset the PHY and perform any required initialization. This can include applying software workarounds for hardware bugs, such as DSP fix ups. Forcing a reset typically causes a momentary link drop, which forces the link partner to also re-sense the link. This is useful for insuring that the link partner actually detects changes made to the PHY media settings.

When attempting to explicitly specify a link speed or duplex setting (rather than using auto-negotiation), Wind River recommends that you specify it while keeping auto-negotiation enabled. While it is possible to disable auto-negotiation and manually configure the PHY for a specific mode, this method can cause problems in certain situations. For example, if the PHY is forced to 100 Mb/s full duplex with auto-negotiation disabled, but the PHY is connected to a link partner that still has auto-negotiation enabled, the link partner uses parallel detection to sense the link speed and default to half duplex. This results in a duplex mismatch that seriously degrades network performance. In addition, manual link configuration is not normally used for 1000 Mb/s links.

A more reliable method is to leave auto-negotiation enabled, but only advertise the particular mode that is desired. For example, to force the link to 10 Mb/s full duplex, the auto-negotiation advertisement register (ANAR) can be programmed to only have the 10FD bit set. Then, the **autoneg** session restart bit is set in the control register. This causes the current PHY and its link partner to agree that 10 Mb/s full duplex is the best common mode for the link. Tests show that this method is fairly interoperable among a wide variety of PHY devices. Consequently, this is the mechanism that the **genericPhy** driver uses.

3. Probe, Attach, and Detach Methods

At a minimum, you must implement probe, attach, and detach methods for your device driver. These methods are required by the VxBus infrastructure.

Probe Method

The PHY driver probe method compares the “match” string of the driver against the PHY ID as provided by the parent MAC driver. If there is no matching driver, the generic PHY driver (**genericPhy**) is used.

Note: Some ARM PHY drivers do not match against the PHY ID provided by the parent, but instead attempt to match against the “compatible” property in the device tree source (DTS) file.

Attach and Detach Methods

The attach method for the PHYs device driver is invoked when the parent device driver (usually a MAC device) creates the PHY child device by calling vxbDevAdd( ).

Before the attach method is called, the MAC driver obtains the PHY address and saves it to the **PHY_HARDWARE** structure from which the PHY driver reads it. An example is as follows:

```
LOCAL STATUS myPhyAttach (VXB_DEV_ID pDev)
{
    MII_DRV_CTRL * pDrvCtrl ;

    ...
    ...

    /* retrieve data set by the parent */
    PHY_HARDWARE * pPhyDev = vxbDevIvarsGet(pDev);
    END_MEDIALIST * pMediaList;
    UINT16 miiSts;

    pDrvCtrl->miiPhyAddr = pPhyDev->phyAddr;
    /* find the parent MAC device */
    pDrvCtrl->pEndInst = vxbDevParent(pDev);

    pDrvCtrl->miiMediaList = pMediaList;
    pDrvCtrl->miiInitialized = TRUE;

    miiBusMediaAdd (pDev, SUPPORTED_MEDIA_TYPE_1);
    ...
    miiBusMediaAdd (pDev, SUPPORTED_MEDIA_TYPE_n);

    /* initialize the PHY hardware */
    miiBusRead (pDev, pDrvCtrl->miiPhyAddr, some_reg, &miiSts);
    miiVal = MY_REG_VALUE;
    miiBusWrite (pDev, pDrvCtrl->miiPhyAddr, some_reg, miiVal);
    ...
    ...
    miiVal = SOME_REG_VALUE;
    miiBusWrite (pDev, pDrvCtrl->miiPhyAddr, some_reg, miiVal);

    /* Add to the monitor list. */
```


WNDRVR

```
    miiDevListAdd (pDev);
}
```

The detach method must remove the PHY device from the MII media list. An example is as follows:

```
LOCAL STATUS genPhyDetach ( VXB_DEV_ID pDev)
{
    VXB_DEV_ID pBus;
    MII_DRV_CTRL * pDrvCtrl;
    UINT16 miiSts;

    pDrvCtrl = (MII_DRV_CTRL *)vxbDevSoftcGet(pDev);

    if (pDrvCtrl->miiInitialized == FALSE)
        return (ERROR);

    /* Only our parent bus can delete us. */
    if (pDrvCtrl->miiLeaving == FALSE)
        return (ERROR);

    /* Remove ourselves from the miiMonitor task list. */
    miiDevListDel (pDev);

    miiBusMediaDel (pDev, SUPPORTED_MEDIA_TYPE_1);
    ...
    miiBusMediaDel (pDev, SUPPORTED_MEDIA_TYPE_n);

    /* Remove media list entries. */
    pBus = vxbDevParent (pDev);
}
```

Shutdown Method

If a detach method is provided by the driver, a shutdown method must be implemented as well in order for the driver to be unloadable. For PHY drivers, the shutdown method can be just a stub that returns OK. For example:

```
LOCAL STATUS genPhyShutdown ( VXB_DEV_ID pDev )
    {
    return (OK);
    }
```

4. PHY Driver Header File Usage

The MII bus APIs and function prototypes are all defined in the **miiBus.h** header file. PHY drivers can also include their own header files.

VxBus PHY drivers (and MAC drivers) must include the **miiBus.h** header file as follows:

```
#include <vxbMiiLib.h>
```

Individual PHY drivers can also include their own header files located in the following directory:

installDir **/vxworks-7/pkgs/net/end/drv/src/mii**

5. Including PHY Driver Support in Your Project

To include PHY driver support in your project, you must add the required component to your VIP.

About This Task

MII bus and PHY instances are autodiscovered (by accessing the MII bus). Therefore, no special BSP configuration is required to add PHY driver support. However, you must include the INCLUDE_GENERICPHY component in your VxWorks image project (VIP).

6. Utility Functions for PHY Drivers

There are two sets of utility functions related to your PHY driver: upper edge functions and lower edge functions. Only lower edge functions are used directly by your PHY driver.

The MII bus module provides both the upper edge functions that are used by Ethernet MAC drivers and the lower edge functions that are used by the PHY drivers themselves. The upper edge functions are typically used to create an MII bus and manage the link. The lower edge functions are used by the PHY drivers to connect themselves to the MII bus subsystem.

Upper Edge Utility Functions

There are a number of upper edge utility functions available to MAC drivers including miiBusModeGet( ), miiBusModeSet( ), and miiBusMediaListGet( ). For information on these functions, see Utility Functions for MAC Drivers on page 12.

Lower Edge Utility Functions

The following lower edge utility functions are available for use by your PHY driver:

miiBusMediaAdd( )

The miiBusMediaAdd( ) function is defined as follows:

```
STATUS miiBusMediaAdd
   (
    VXB_DEV_ID pDev,
    UINT32 media
    )
```

This function is used by PHY drivers to notify the MII bus about the media types that they support. The function is normally called by a PHY driver's initialization code, and is used to populate the media list information that is returned by the miiBusMediaListGet( ) function described in miiBusMediaListGet( ). A typical 10/100 Ethernet PHY may specify its media support as shown in the following example:

```
miiBusMediaAdd (pBus, IFM_ETHER|IFM_100_TX);

miiBusMediaAdd (pBus, IFM_ETHER|IFM_100_TX|IFM_FDX);

miiBusMediaAdd (pBus, IFM_ETHER|IFM_10_T);

miiBusMediaAdd (pBus, IFM_ETHER|IFM_10_T|IFM_FDX);

miiBusMediaAdd (pBus, IFM_ETHER|IFM_AUTO);
```

miiBusMediaDel( )

The miiBusMediaDel( ) function is defined as follows:

```
STATUS miiBusMediaDel
    (
    VXB_DEV_ID pDev,
    UINT32 media
    )
```

This function is used by PHY drivers to remove their supported media types from their parent bus' media list when the device is unloaded.

miiBusMediaDefaultSet( )

The miiBusMediaDefaultSet( ) function is defined as follows:

```
STATUS miiBusMediaDefaultSet
    (
    VXB_DEV_ID pDev,
    UINT32 media
    )
```

This function is used by PHY drivers to specify the default media selection that is listed in the media list for a given bus. Typically, the default media type is IFM_AUTO.

miiBusRead( )

The miiBusRead( ) function is defined as follows:

```
STATUS miiBusRead
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 phyReg,
    UINT16 * regVal
    )
```

This function is used by a PHY instance to read its own registers. The pDev argument is a pointer to the parent MII bus. This function in turn invokes the {miiRead}( ) method exported by the parent Ethernet MAC driver or by the parent MDIO driver.

miiBusWrite( )

The miiBusWrite( ) function is defined as follows:

```
STATUS miiBusWrite
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 phyReg,
    UINT16 regVal
    )
```

This function is used by a PHY instance to write its own registers. The pDev argument is a pointer to the parent MII bus. This function in turn invokes the {miiWrite}( ) method exported by the parent Ethernet MAC driver or by the parent MDIO driver.

Note: Register values are always in native byte order.

miiBus45Read( )

The miiBus45Read( ) function is defined as follows:

```
STATUS miiBus45Read
        (
        VXB_DEV_ID pDev,
        UINT8 phyAddr,
        UINT8 devAddr,
        UINT16 phyReg,
        UINT16 * regVal
        )
```

This function is similer to miiBusRead( ) except that it is used for clause 45 PHY devices. This includes 10 gigabit ethernet PHYs, and similar high speed transceivers. The devAddr argument is a value from 0 to 31 which specifies which sub-device to access when performing the read.

miiBus45Write( )

The miiBus45Write( ) function is defined as follows:

```
STATUS miiBus45Write
        (
        VXB_DEV_ID pDev,
        UINT8 phyAddr,
        UINT8 devAddr,
        UINT16 phyReg,
        UINT16 regVal
        )
```

This function is similer to miiBusWrite( ) except that it is used for clause 45 PHY devices. This includes 10 gigabit ethernet PHYs, and similar high speed transceivers. The devAddr argument is a value from 0 to 31 which specifies which sub-device to access when performing the write.

7. VXB_DRV Structure for PHY Drivers

A VxBus device driver must include a driver declaration structure of type **VXB_DRV** which is used by VxBus during driver registration.

The **VXB_DRV** structure for PHY drivers is as follows:

```
VXB_DRV  myPhyDrv =
    {
    { NULL } ,
    "myPhy",            /* drvName */
    "my Phy driver",    /* Description */
    VXB_BUSID_MII,      /* Class */
    0,                  /* Flags */
    0,                  /* Reference count */
    myPhyMethods        /* Method table */
    };

VXB_DRV_DEF(myPhyDrv)
```

Note: The VxBus bus type (class) is typically VXB_BUSID_MII.

4. MDIO DRIVERS

When a media independent interface (MII) bus is used as an interface between an Ethernet controller and a PHY chip, you must provide a VxBus management data input/output (MDIO) device driver.

MDIO is defined in clause 22 and clause 45 of the IEEE 802.3 specification. It is a 2-wire serial bus that is used to manage the PHYs or physical layer devices of media access controllers (MACs) in Ethernet equipment.

One clause 22 MDIO interface can support up to 32 registers, in 32 different PHY devices. One clause 45 MDIO interface can support up to 32 devices, each with 32 sub-devices, and 65536 registers per sub-device. The registers provide status and control information such as link status, speed ability and selection, power down for low power consumption, duplex mode (full or half), autonegotiation, fault signaling, and loopback.

The PHY device driver, as well as the **miiBusMonitor** task, invoke the MDIO driver read and write methods (by calling the {miiRead}( ) and {miiWrite}( ) methods, or the {mii45Read}( ) and {mii45Write}( ) methods) to access the PHY chip.

Therefore, the MDIO read and write methods are responsible for access arbitration to the single MDIO port.

VxBus Driver Methods

Your MDIO device driver must provide the following methods:

probe( )

This method matches the device driver against a compatible name in the device tree source (DTS) file.

attach( )

This method allocates a memory resource to enable access to the MDIO registers. It can also create the semaphores used for access control to the MDIO registers. Depending on the actual hardware, certain hardware initialization must be performed here as well. Depending on your VxBus topology, the MDIO device may be the parent of the PHY device. In this case the attach( ) method invokes the PHY's attach method.

shutdown( )

This method can be used to shut down the hardware. Since it may be used as a reboot hook, it must not make any blocking calls. In most cases it may be safe to simply make it a stub that returns OK, since MDIO devices do not usually trigger interrupts. If a detach method is provided, a shutdown method must be provided as well (even if it is just a stub), in order for driver unloading to work correctly.

detach( )

This method performs house cleaning such as deleting semaphores.

{miiRead}( )

The {miiRead}( ) driver method allows PHYs on the MII bus to access your device's MII management registers. This method has the same signature as the following stub routine:

```
LOCAL STATUS miiRead
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 regAddr,
    UINT16 * dataVal
    )
    {
    return (OK);
    }
```

Each MAC driver must provide an MII bus read function so that the MII bus code can perform management data input/output (MDIO) read accesses to connected PHYs.

The **phyAddr** argument indicates which PHY address is to be queried, and can be any value from 0 to 31. The **regAddr** argument indicates which register is to be read, and can also be any value from 0 to 31. The dataVal( ) argument points to a 16-bit storage location where the func{miiRead}( ) method places the value read from the specified register. If the func{miiRead}( ) method fails and returns **ERROR**, **dataVal** is set to 0xFFFF.

It is possible for the {miiRead}( ) method to return successfully but not return any valid data. For example, if a request is made to read register 1 on the PHY at address 10, but there is no PHY actually available at that address, then the MDIO access may succeed, but no valid register information is obtained. In this case, the hardware typically returns a value of 0xFFFF. The MII bus probe code checks for this case and only assumes that valid data is returned if the {miiRead}( ) method succeeds and the register value is not 0xFFFF.

{miiWrite}( )

The {miiWrite}( ) driver method allows PHYs on the MII bus to access the device MII management registers. The {miiWrite}( ) method has the same signature as the following stub routine:

```
LOCAL STATUS miiWrite
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 regAddr,
    UINT16 dataVal
    )
    {
    return (OK);
    }
```

The {miiWrite}( ) method is the complement to the {miiRead}( ) method, allowing the MII bus to perform MDIO write accesses to connected PHYs. The **phyAddr** and **regAddr** arguments are the same as they are for {miiRead}( ). The **dataVal** argument is the value to be written into the register. The func{miiWrite}( ) function must only return OK if the register is successfully updated.

{mii45Read}( )

The {mii45Read}( ) driver method is used to perform a PHY register read using the clause 45 MDIO protocol. It is similar to the {miiRead}( ) method, except that it takes an additional devAddr argument to specify which sub-device to access. Like phyAddr, regAddr should be a value from 0 to 31. Also, the register argument is 16 bits wide instead of 8, since each sub-device can have up to 65536 registers.

```
LOCAL STATUS mii45Read
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 devAddr,
    UINT16 regAddr,
    UINT16 * dataVal
    )
    {
    return (OK);
    }
```

{mii45Write}( )

The {mii45Write}( ) driver method is used to perform a PHY write using the clause 45 MDIO protocol. It's similar to the {miiWrite}( ) method, except that it takes an additional devAddr argument to specify which sub-device to access. Like phyAddr, regAddr should be a value from 0 to 31. Also, the register argument is 16 bits wide instead of 8, since each sub-device can have up to 65536 registers.

```
LOCAL STATUS mii45Write
    (
    VXB_DEV_ID pDev,
    UINT8 phyAddr,
    UINT8 devAddr,
    UINT16 regAddr,
    UINT16 dataVal
    )
    {
    return (OK);
    }
```

MII Utility Routines

MDIO drivers are allowed to use the following **vxbMiiLib** library utility routines:

miiBusCreate( )

The miiBusCreate() function is defined as follows:

```
STATUS miiBusCreate
        (
        VXB_DEV_ID pBus,
        UINT32 flags
        )
```

The miiBusCreate( ) function is used to instantiate PHY device instances and attach them to a parent MDIO bus. The parent can be any VxBus node that implements the {mdioRead}( ) and {mdioWrite}( ), or the {mdio45Read}( ) and {mdio45Write}( ), methods. The miiBusCreate( ) function automatically will probe PHYs by performing an MII read of the status register at every PHY address from 0 to 31. Any linkages between PHYs and MACs that are specified in the device tree file in the BSP will be honored.

The flags argument specifies any special properties for the MDIO bus. Currently only two flags are supported: **VXB_MDIO_FLAG_CLAUSE22** and **VXB_MDIO_FLAG_CLAUSE45**. These flags are mutually exclusive: a single MDIO parent should not have both clause 22 and clause 45 devices present on the same two-wire management bus. If an MDIO instance has clause 22 PHYs, it should use the **VXB_MDIO_FLAG_CLAUSE22** flag. For MDIO instances that manage clause 45 devices, it should use **VXB_MDIO_FLAG_CLAUSE45**. Note that it is possible for one driver to support both kinds of devices, but not at the same time.

For example, the MEMAC MDIO driver used with T2080 and T4240 processors can select either clause 22 or clause 45 mode depending on how the system is configured. However each MDIO instance is configured for one more or the other at start-up and can not switch modes on the fly.

When a PHY node is added, the MAC parent of the PHY will have its {miiMediaAttach}( ) method called to notify it of the new device's existence.

miiBusDelete( )

The miiBusDelete( ) function is defined as follows:

```
 STATUS miiBusDelete
        (
        VXB_DEV_ID pBus
        )
```

This function is used by an MDIO parent to delete all the child PHY devices that it manages. The underlying device nodes will be removed from the VxBus tree. If the parent MAC of a given PHY supports the {miiMediaDetach}( ) method, it will be called

WNDRVR

so that it knows the PHY device is being removed. (This handles the situation in which the MAC parent and MDIO parent of a given PHY are not the same device.)